FABLE: Cost-Effective High-Concurrency LLM Inference via Fine-Grained Prefetching and Filesystem Bypass

Wen-Jie Luo

Abstract—Deploying large language models (LLMs) on a single consumer-grade GPU is limited by both GPU memory capacity and host DRAM size. Utilizing SSD to host model weights and KV caches circumvents GPU memory and host DRAM limitations. However, this often causes model inference to become I/O-bound. With layer granularity prefetching, computation still waits for whole layer transfers and the GPU stalls. The conventional SSD to DRAM path traverses the filesystem and OS page cache and incurs redundant memory copies that increase I/O latency and reduce throughput. We present FABLE, short for Fine-grained prefetching And filesystem Bypass for LLM inferencE, which combines sublayer granularity prefetching with a filesystem bypass SSD path using direct I/O. FABLE decomposes each layer into schedulable MHA and FFN units and transfers unit sized objects. An MHA object bundles attention weights with the per request KV cache, while an FFN object contains only FFN weights. By overlapping object transfers with unit execution, FABLE reduces GPU stalls, and direct I/O eliminates redundant copies during SSD to DRAM transfers. Serving BF16 Meta LLaMA 3.1 70B on an RTX 5080, FABLE reduces end to end latency by up to 66.2 percent, improves decoding throughput by up to 3.63 times, and reduces time to first token by up to 41.6 percent versus on demand offloading through filesystem I/O. Under the same bypass path, FABLE improves decoding throughput over FlexGen-style by 1.27 times at batch size 64.

Index Terms— Large language models, KV cache, Hierarchical storage, Filesystem

1. INTRODUCTION

In recent years, large language models (LLMs) have been widely adopted in many applications, particularly for generative workloads [2, 19, 38]. At inference time, which is the process of generating tokens from a trained model, these systems often require powerful GPUs, which makes deployment and operation expensive. As generative services scale, improving cost efficiency requires increasing throughput per GPU, i.e., the token-serving capacity delivered by one GPU under a fixed hardware budget [30, 42].

To improve generation quality, modern LLMs have grown to very large parameter counts [19], which increases the memory and computation resources required for inference. Meanwhile, long prompts and high-concurrency serving, where the engine processes many requests in parallel, have become common in production [38]. These trends increase the inference-time memory footprint, including model weights (the learned parameters of the model) and inference runtime states such as activations and the key–value (KV) cache. The KV cache stores key and value vectors from prior tokens so that attention can reuse past context without recomputation [5, 6]. Inference engines typically keep KV cache in GPU device memory, which we refer to as GPU memory in this paper for simplicity. Meeting these memory demands by scaling out to many GPUs can be prohibitively expensive. As a result, many recent inference systems adopt a heterogeneous memory hierarchy spanning GPU memory, host DRAM and NVMe SSD by offloading model state. In this approach, weights and the KV cache are stored in lower-cost host DRAM or NVMe SSD and are moved to the GPU on demand which inference namely hierarchical storage system [3, 4, 10, 11, 13, 14, 15, 23, 29].

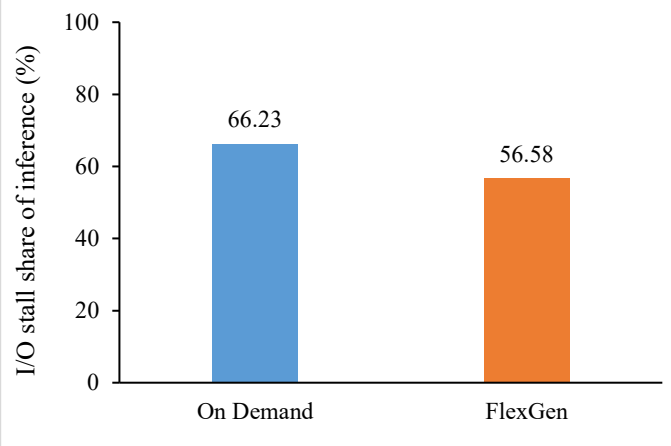


Figure 1. I/O Stall Share of Inference Latency

Although hierarchical offloading expands effective capacity, it also exposes large bandwidth and latency gaps across GPU memory, host DRAM, and SSD.When model weights or KV cache reside in DRAM or SSD, inference become I/O bound because execution must repeatedly wait for these data to be transferred into GPU memory. In this regime, inference latency is dominated by data movement rather than GPU computation, leaving the GPU idle while the inference engine waits for inputs to arrive. FlexGen [3] is a hierarchical offloading system which uses prefetching to transfer data ahead of use so that data movement can overlap with useful computation to reduce this latency. Its benefit is limited by compute slack, the compute time available before the prefetched data is needed. Any transfer time that exceeds this slack still becomes a stall. Prefetching is typically performed at whole transformer layer granularity as the smallest scheduling task unit. GPU have to wait until the whole layer's data have been loaded into GPU memory before computation can begin, which increase GPU waiting latency. As Figure 1 show, under whole-layer granularity, the on demand hierarchical offloading baseline without prefetching and FlexGen[3] shares of inference latency when serving LLaMA 3.1 70B with 2,048-token prompts, batch size 64, and a generation length of 32 tokens. In the on-demand system, 66.23% of inference latency is spent in I/O stalls, meaning the GPU waits for data for more than half of the end-to-end inference time. Even with prefetching, FlexGen still spends 56.58% of inference time in I/O stalls when operating at whole-layer granularity. This indicates that prefetch under whole-layer granularity continues to waste GPU time by lowering GPU utilization and thus increasing inference latency. Meanwhile, weights and KV caches still traverse NVMe and PCIe links, this data movement can dominate latency [28]. We

also observe SSD to DRAM transfers are often served through the conventional I/O path, which traverses filesystem and the OS page cache. This path incurs extra overhead for cache management and frequently introduces an additional copy between kernel buffers and user buffers, thereby increasing end-to-end transfer time. As a result, these observations highlight two primary challenges in current offloading systems: whole-layer-granularity prefetching still leaves substantial I/O stalls that waste GPU time and increase inference latency, and SSD-to-DRAM transfers often go through the conventional I/O path extra copies that inflate data-movement latency.

To address these challenges, this paper proposes FABLE, short for Fine-grained prefetching And filesystem Bypass for LLM inferencE. The name reflects two concrete design choices in this paper. First, fine-grained prefetching that FABLE does not treat a whole transformer layer ℓ as the minimum scheduling unit. Specifically, layer ℓ scheduling unit decomposes into two fine-grained units: multi-head self-attention (MHA_ℓ) and a feed-forward network (FFN_ℓ). For each unit, FABLE defines MHA_ℓ object which includes the MHA_ℓ unit's weights of layer $\ell$and the KV cache associated with layer ℓ and FFN_ℓ object which contains the FFN unit's weights of layer ℓ. FABLE transfers at object granularity, also referred to as sublayer granularity. FABLE prefetches the next object data ahead of use and pipelines SSD-to-DRAM and DRAM-to-GPU memory transfers with current execution unit using CUDA streams and events. Second, with filesystem bypass, FABLE accesses NVMe through a direct path that bypass the filesystem and page cache[28], reducing redundant copying and improving the efficiency SSD to DRAM transfers. With these mechanisms, FABLE targets cost-efficient, high-concurrency inference under long prompts [37] for large models such as LLaMA 3.1 70B [35] on a single consumer-grade GPU. To validate the correctness of the FABLE engine, we conducted a sanity-check experiment using the same model. We ran the same prompt with the identical model checkpoint, inputs, and decoding settings on FABLE and on a reference implementation based on the Hugging Face Transformers library [49]. The generated outputs were highly similar, while FABLE achieved substantially better end-to-end performance, indicating that FABLE preserves the expected model behavior and behaves as intended.

We evaluate FABLE by serving LLaMA 3.1 70B on a fixed single-machine testbed and measuring three user and cost relevant metrics under long prompts and high request concurrency: end-to-end latency, the wall-clock time from request arrival to the last output token; time-to-first-token (TTFT), which captures the initial responsiveness before the first token is streamed; and throughput, measured in tokens/s, which determines GPU serving capacity and cost efficiency. Compared with on demand offloading baseline that uses filesystem I/O, FABLE reduces end-to-end latency by up to 66.2%, improves decoding throughput by up to 3.63×and reduces time to first token by up to 41.6%. Compared with FlexGen-style, a representative system that serves large models by moving weights between GPU memory, host DRAM, and SSD at inference runtime under the FABLE same filesystem-bypass I/O path, FABLE reduces end-to-end latency by about 15.3% and improves decoding throughput by about 1.27×at batch size 64. Ablations show that the filesystem-bypass path alone improves decoding throughput by 1.93×over filesystem I/O. These results indicate that combining fine-grained prefetching with a filesystem-bypass path can materially improve throughput GPU in storage-constrained deployments. To summarize, this paper makes the following contributions:

- We design and implement an inference engine that enables fine-grained prefetching at object granularity and overlaps computation with multi-tier data movement to support large-model inference on a single GPU (Section 3.2).
- We build a filesystem-bypass I/O path that accelerates SSD-to-DRAM transfers and reduces storage-stack overhead from the filesystem and page cache (Section 3.3).

2. BACKGROUND AND RELATED WORK

This chapter provides the background needed to understand storage-constrained large language models (LLMs) inference. We first summarize the Transformer-based, autoregressive inference workflow and its two-phase structure (prefilling and decoding). We then explain why heterogeneous memory hierarchies (GPU memory, host DRAM, and NVMe SSD) are attractive but often become I/O-bound which is data transfers take most of the time, so the GPU often sits idle while waiting for data in practice. Finally, we review representative systems and algorithmic techniques.

2.1. Fundamentals of Generative LLM Inference

Transformer architecture [1, 12, 30, 34] is the standard backbone for modern LLMs. Mainstream model families such as GPT [2, 33] and LLaMA [18, 19, 37] use an autoregressive Transformer. Autoregressive means that the model generates tokens one by one, and each new token is conditioned on all previous tokens. At inference time, the system takes a user prompt and produces a response. A prompt is the input text provided by the user. The system first runs a tokenizer to convert the prompt into a sequence of tokens. A token is an integer ID in a fixed vocabulary. The system then maps each token to a tensor, which is a multi-dimensional numeric array that has a fixed element type (dtype) and shape are typecially stored in GPU memory. Model operators consume one or more input tensors and produce one or more output tensors.

A Transformer [1] model contains L stacked layers. Figure 2 gives a high-level view of inference: each dashed box denotes one forward pass through the full layer stack. A layer is the basic repeated block in the network. Each layer has two calculation processes. The first process is Multi-Head Self-Attention (MHA). MHA lets each token attend to other tokens in the same sequence. The second process is a Feed-Forward Network (FFN). FFN applies position-wise nonlinear transformations to each token representation. Each process has its own weights. Weights are learned parameters stored as matrices. In layer ℓ, MHA uses projection matrices $\{W_Q^\ell, W_K^\ell, W_V^\ell, W_O^\ell\}$ where W_Q, W_K, and $W_V$project activations into queries, keys, and values for attention, and $W_O$projects the attention output back to the model dimension. FFN uses matrices such as $\{W_1^\ell, W_2^\ell, W_3^\ell\}$ to implement the per-token feed-forward transform. In many modern LLMs, this is a gated MLP, e.g., GLU/SwiGLU that consists of three linear

projections, often called the gate, up, and down projections. The input tensor enters MHA first. MHA produces an intermediate tensor. FFN consumes this intermediate tensor and produces layer output. The model processes layers sequentially from $\ell = 1$ to L, so N = L in Figure 2. After the last layer, the model outputs a probability distribution over the vocabulary. The system uses this distribution to select the next token.

In self-attention at layer ℓ, each token x hidden state is linearly projected into a query q, a key k, and a value v(e.g., $q = xW_Q^\ell$, $k = xW_K^\ell$, $v = xW_V^\ell$)[1]. This section calls one (K, V) pair for one token in one layer a KV pair. During prefilling, the model generates KV pairs for all prompt tokens. During decoding, the current token must attend to the keys and values of all previous tokens. Without caching, the model would need to recompute these past keys and values at every step. To avoid this repeated work, inference engines store KV pairs in a KV cache [6, 22, 30]. A KV cache is the collection of per-layer KV pairs for previously processed tokens as Figure 2 every decoding layer will receive the before layer KV cache. The KV cache is usually stored in GPU memory for speed. The KV cache grows with both sequence length and the number of concurrent sequences. The KV cache grows linearly with tokens and can become very large. For example, GPT3 [2, 33] generates a 4.5MB KV cache for each token. A long prompt such as 2,048 tokens can therefore consume multiple gigabytes of GPU memory for KV alone.

Figure 2 illustrates the inference flow. The generation process can be divided into two phases: prefilling and decoding [24]. This two-phase view is widely used in systems work because the phases have different behavior. In the prefilling phase, the model processes the entire prompt $[x_1, x_2, \ldots, x_s]$. For example, a prompt such as "Is LLM good?" may be tokenized into a sequence such as "Is","LLM","$good$","?"and fed into layer 0 as the initial input. The system computes hidden states for all prompt tokens. It also materializes the KV cache for these tokens. This KV cache will be reused in the next phase. Prefilling can compute many prompt tokens in parallel. In the decoding phase, the model generates output tokens one by one such as Figure 2 generate "it" first, then generate "Is" next. The computation proceeds sequentially, one time step at a time. At each step, the model takes the current token and attends to the KV cache of all previous tokens to produce the next token and its KV pair. The system stops when it generates the end-of-sequence token <EOS> or reaches a maximum generation length. <EOS> is a special token that marks completion.

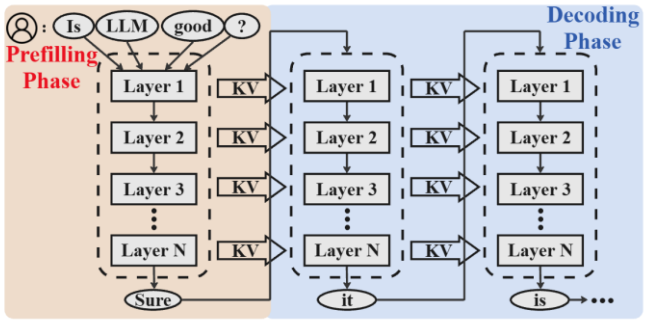


Figure 2. General Architecture of LLM and Inference Flow

Some inference engines implement chunked prefilling like vllm, SGLang [6, 27] as a variant of the prefilling phase. Chunked prefilling splits a long prompt into multiple chunks and processes these chunks sequentially while incrementally extending the KV cache. Chunked prefilling limits how many prompt tokens are processed at once, so it reduces the peak GPU memory required during prefilling, where peak GPU memory is the maximum GPU memory used at any time during execution. This helps avoid out-of-memory during prefilling. It does not emit output tokens during chunked prefilling because each chunk only builds part of the prompt KV cache, and token generation begins only after the full prompt context has been encoded. The engine enters the decoding phase, and thus emits output tokens for that request, only after all prompt chunks have been processed, because decoding requires the KV cache for the full prompt.

However, chunked prefilling does not change the model weight size, and the KV cache still grows with context length and concurrency in decoding; therefore, GPU memory pressure can still lead to out-of-memory in practice.

2.2. Resources and Bottlenecks in LLM Inference

In contemporary LLM families such as LLaMA [18, 19, 37] and GPT [2], model parameter counts continue to grow. This growth increases the storage footprint of weight tensors. Consider LLaMA-70B [35]. The model has 70 billion parameters. Under BF16 precision [16, 21] without quantization, the weights alone require about 140 GB of storage. A single GPU provides far less GPU memory. For example, NVIDIA A100 provides 80 GB GPU memory [31]. NVIDIA H100 also provides 80 GB GPU memory [32]. Therefore, the weights alone require multiple GPUs. Beyond the static weight footprint, inference requires additional memory, most notably the KV cache, whose footprint grows with context length and concurrency. In high concurrency serving, KV cache memory grows linearly with sequence length and the number of concurrent sequences (requests processed in parallel) [42]. During prefilling, the engine allocates KV entries for all prompt tokens, so longer prompts allocate proportionally more KV cache. During decoding, each newly generated token appends new KV entries, further growing the KV cache footprint. A single model instance can require many 80GB GPUs to inference [7, 23].

Offloading-based inference systems extend GPU memory with a heterogeneous memory hierarchy. Such systems store the model state (weights or KV cache) in host DRAM (CPU main memory) or NVMe SSD (PCIe-attached flash storage). For example, vLLM [6] provides options to offload part of the KV cache to CPU memory. Its documentation describes this as a virtual increase in available GPU memory per GPU. ZeRO-Inference[45] pins the full model weights in host DRAM and streams each layer's weights into GPU memory on demand, releasing the layer weights after the layer finishes. The main benefit is higher effective memory capacity: offloading enables larger models, longer contexts, or more concurrent sequences with fewer GPUs, and can reduce hardware cost because DRAM and SSD are significantly cheaper than GPU memory.

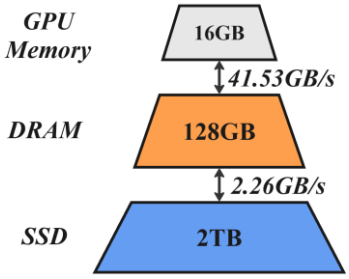


Figure 3. Storage Hierarchy with Bandwidth & Size

Offloading increases usable memory capacity, but it shifts time from GPU computation to cross-tier transfers. Bandwidth and latency differ by orders of magnitude among GPU memory, host DRAM, and NVMe SSD [9, 17]. We measure effective bandwidth along the baseline paths in our testbed. We use a CUDA memcpy microbenchmark [41] with pinned host buffers for host DRAM-to-GPU transfer. We use a streaming file-read test with a cold page cache for NVMe SSD-to-host DRAM transfer. In Figure 3, host DRAM-to-GPU reaches 41.53 GB/s, while NVMe SSD-to-host DRAM reaches 2.26 GB/s. This gap can block kernel launches when tensors are fetched on demand data. This effect is especially visible in decoding, which is often I/O-bound that provides limited compute slack to hide cross-tier transfers. It can leave the GPU idle and increase inference latency.

Prior work uses three common ways to make offloading more practical under this bandwidth gap. First, quantization reduces tensor size by storing weights or KV cache in lower precision [16, 21]. This reduces transfer volume. However, it can introduce accuracy concerns which may degrade output quality and task performance, and compression or decompression overhead. Second, memory-management techniques reduce waste and improve allocation and reuse in GPU memory and host DRAM [6, 8]. However, it adds bookkeeping overhead and does not reduce the cost of transferring data across tiers This increases the effective capacity of each tier. Third, prefetching and related overlap techniques transfer data ahead of use [3, 11]. However, it benefits is limited by the available compute slack. These approaches hide part of the transfer latency when computation provides slack. In practice, two limitations remain. First, many of the offloading engines use whole transformer layer granularity dependencies for weights, so a layer cannot start until its full weights and the required KV cache arrives, so the GPU stall for data regardless of the technique used. (e.g., FlexGen[3] uses layer granularity for weights). Second, many systems do not model or analyze the SSD read path in detail. The conventional filesystem path can add extra copies and less predictable latency. To our knowledge, prior LLM offloading systems do not address both gaps together for steady-state inference, which motivates Pro1 and Pro2.

Problem 1 (Pro1): Coarse-grained scheduling can lead to low GPU utilization. Many of the offloading designs prefetch weights at whole-layer granularity, yet the GPU cannot start layer until all of its required weights are present in GPU memory. In a host DRAM or SSD-offloaded setup, the SSD to host DRAM to GPU transfer latency can exceed the computation time of a single layer. As a result, the GPU often finishes the current layer before the next layer's weight arrives and becomes idle. This idle period forms a pipeline bubble, namely a stall gap where the GPU has no ready work. I/O latency jitter on the transfer path can further enlarge these bubbles and make stall time more variable.

Problem 2 (Pro2): The conventional storage stack adds inference latency. Many systems load data from NVMe SSD through the filesystem. With conventional filesystem I/O, data is first brought into the OS page cache which is the kernel buffer for file data, and then copied by the CPU into a user-space buffer. Then filesystem copies it into a user-space buffer. For GPU direct memory access (DMA), the filesystem often needs a pinned buffer. A pinned buffer is host DRAM that the GPU can access by DMA. The filesystem may copy it again into pinned memory. These extra copies add CPU work and extra memory traffic. It can also increase latency variability due to page-cache management overhead and contention under memory pressure. This inflates SSD to host DRAM transfer time which increases inference latency by GPU stall while data is in transit.

2.4 Related work

KV Cache Management. The key–value (KV) cache is a per-request inference runtime memory that stores past attention "keys" and "values" so the model can reuse prior context. It grows with prompt length and request size. Many systems aim to reduce the KV cache footprint or make it easier to allocate and reuse. vLLM [6] manages the KV cache in fixed-size "pages" (similar to OS paging) so memory does not need to be reserved as one large contiguous chunk, reducing waste and improving allocation flexibility. SGLang [27] introduces RadixAttention, which uses a prefix-tree (radix tree) index to reuse KV cache across requests or multi-call programs that share the same prompt prefix. H2O [11] proposes a rule for discarding KV entries (an eviction policy): it keeps a mix of the most recent tokens and a small set of "important" tokens that are frequently attended to. KVQuant [16] compresses KV cache activations by storing them in fewer bits, enabling longer context with small quality degradation. InfiniGen [13] targets long-text generation and reduces offloading overhead by predicting which KV entries will be needed soon and prefetching only those entries rather than fetching the full KV cache. FlashAttention [8] accelerates attention on GPU by processing attention in small blocks (tiling) to reduce expensive GPU-memory reads or writes. Overall, these works mainly reduce KV size, waste, or on-GPU attention traffic, whereas FABLE targets GPU stalls and transfer overhead when weights or KV cache must move across DRAM or SSD, making it complementary to these techniques.

Offloading over a Heterogeneous Memory Hierarchy. Another line of work makes GPU memory act "larger" by offloading model state to cheaper tiers: host DRAM and SSD. FlexGen [3] is a representative baseline that runs large models on a single GPU by combining GPU memory, host DRAM, and SSD and compressing both weights and the attention cache. DeepSpeed-Inference [5] provides a heterogeneous inference mode that can store model state in CPU memory and NVMe when it does not fit in GPU memory. M2Cache [12] combines mixed precision with a three-level cache (GPU memory / host DRAM / SSD) so frequently used parts stay closer to the GPU while the full model can reside on SSD. CachedAttention [4]

targets multi-turn conversations by saving and reusing KV caches across hierarchical stores, and it overlaps KV movement with compute using layer-wise pre-loading and asynchronous saving. Mooncake [23] separates prefilling phase from decoding phase and builds a KV-cache-centric disaggregated architecture and scheduler that leverage CPU/DRAM/SSD resources in a cluster. InstInfer [15] offload attention computation and KV handling to computational storage drives, reducing PCIe transfers for long-context inference. NVIDIA also recently introduced a BlueField-4-powered Inference Context Memory Storage platform that treats KV cache as a first-class "context memory" tier, using a network-attached flash layer and DPU offloads to accelerate KV cache access or sharing and reduce host to side copies and decode stalls [48]. While sharing the same goal as these systems—making DRAM or SSD a practical extension of GPU memory for inference state—FABLE focuses on two remaining gaps in SSD/DRAM offloading: coarse layer-level that create GPU bubbles, and conventional filesystem-based SSD paths that add copies and latency variability. FABLE addresses them with fine-grained prefetching and filesystem bypass.

3. DESIGN AND IMPLEMENTATION

3.1. Overview

This chapter presents the design of FABLE, a single-GPU inference system that data movement and execution while serving large LLMs under long prompts and high concurrency where the engine processes many requests in parallel. We assume a three-tier hierarchy: NVMe SSD, host DRAM, and GPU memory. FABLE targets a commodity workstation with one consumer-grade GPU and moderate host DRAM (our testbed has 16 GB GPU memory and 128 GB host DRAM). In this setting, the combined memory demand of model weights and the KV cache under long prompts and many concurrent sequences can exceed host DRAM capacity. Therefore, FABLE stores the weights and KV cache on NVMe SSD. FABLE uses host DRAM as a staging cache to stream data to GPU memory.

To avoid ambiguity, we define the terms used in this section. Each transformer layer ℓ has two main units: multi-head self-attention (MHA_ℓ) and a feed-forward network (FFN_ℓ) [1] which is the minimum execution unit that FABLE schedules. For each layer ℓ, FABLE defines an MHA_l object that contains the weights needed by the MHA_ℓunit and the layer-ℓ KV cache, as well as an FFN_ℓobject that contains the weights needed by the FFN_lunit. Similar SGLang[27], FABLE stores attention state in a KV cache and partitions it into fixed-size KV blocks, following block-based KV organizations used in serving systems. Each KV block covers 512 tokens for one request at one layer (B=512 in our implementation). SSD serves as the store of KV blocks. We use promote/demote for bidirectional data movement between SSD and DRAM and prefetch/offload for bidirectional data movement between DRAM and GPU memory. Newly produced KV blocks are first staged in the DRAM and then demoted to SSD. Before executing MHA at layer ℓ, FABLE promotes the required KV blocks from SSD into the DRAM according to the fine-grained prefetch schedule. A tier is one level in the SSD to DRAM to GPU memory hierarchy. FABLE maintains a sliding-window working set, namely the resident objects to launching the next units. FABLE bounds data residency with GPU MAX object and CPU MAX object, which cap the number of resident objects in GPU memory and host DRAM.

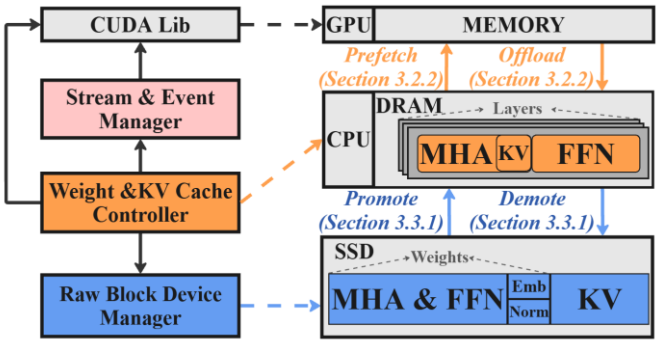


Figure 4. System Architecture of FABLE

Figure 4 summarizes FABLE as three components that connect filesystem-bypass I/O, multi-tier caching, and GPU execution into one compute–I/O pipeline, where data transfers are scheduled ahead of use. Solid arrows denote bulk data movement across tiers, while dashed arrows denote interface relationships between components and the tiers manage. Direct I/O bypasses the filesystem and OS page cache, while imposes strict alignment constraints on the buffer address, I/O offset, and I/O size. FABLE therefore encapsulates all NVMe direct-I/O operations in the Raw Block Device Manager, which enforces block-aligned reads and writes and hides the alignment details from the rest of the runtime. The controller interacts with this manager to issue promote and demote requests and to obtain the requested object. Concretely, the manager issues block-aligned positional reads and writes to the raw NVMe device and serves each object's weights and SSD-resident KV blocks. Each object's weights are stored as a few contiguous byte ranges on the device, which the manager reads into block-aligned, page-locked (pinned) DRAM staging buffers. Pinned buffers enable subsequent host-to-device transfers to use DMA efficiently and overlap with kernel execution. The Weight & KV Cache Controller is the policy and bookkeeping layer. It tracks residency of objects across SSD, host DRAM, and GPU memory, and maintains a sliding-window working set that stays ahead of execution. This is why the controller connects upward to both the Stream & Event Manager and the CUDA library: the controller not only decides what to move and when, but also directly enqueues the corresponding host DRAM to GPU memory copy operations onto the appropriate CUDA streams, while delegating stream provisioning and event-based dependency wiring to the Stream & Event Manager. Concretely, the controller chooses the lookahead distance and the next objects to stage, enforces capacity budgets (CPU MAX object / GPU MAX object) and triggers evictions or write-back when space are full, and updates per-object state so that data movement is issued early but never exceeds the DRAM/GPU working-set limits. The Stream & Event Manager is the execution layer that turns the controller's decisions into GPU progress. It connects to the CUDA library because CUDA streams and events provide the mechanism to provision dedicated work queues for compute and data movement and

express cross-stream dependencies without global synchronization. Concretely, the manager creates and owns a fixed stream/event layout (e.g., separate compute streams for MHA_ℓ and FFN_ℓ that separate transfer streams for MHA, FFN and KV cache movement), and exports the corresponding stream handles to the controller so the controller can enqueue host to device copies onto the intended transfer streams. The manager then wires dependencies by recording "ready" events after the relevant copies in the transfer streams and inserting stream-wait operations before the dependent kernels in the compute streams; it also enforces the intended compute order across MHA_ℓ and FFN_ℓ via events when needed. This event-based wiring enables I/O–compute overlap and allows each sublayer to launch as soon as its own inputs become available, without global synchronization.

To address Pro1, instead of waiting for all of a layer's weights to arrive before launching computation, FABLE splits each Transformer layer ℓ into two schedulable units, MHA_ℓ and FFN_ℓ. It packages each unit's required inputs into a corresponding object and tracks readiness with per-unit events (e.g., weight-ready and KV-ready). This directly addresses the case where SSD to DRAM to GPU memory transfer latency exceeds the computation time of a single layer: the controller maintains a lookahead sliding window and continuously promotes/prefetches upcoming objects while the GPU executes the current unit, overlapping transfers with kernels on separate CUDA streams. By ensuring that FFN_ℓ object is staged during MHA_ℓ execution and that $\mathrm{MHA}_{\ell+1}$ object is staged during FFN_ℓ execution, FABLE reduces pipeline bubbles where the GPU stall at layer boundaries. Finally, to mitigate I/O jitter that would otherwise enlarge these bubbles, FABLE makes stalls local and schedulable: compute streams wait only on the specific transfers they depend on (via CUDA event record/wait), and transfers use pinned host buffers for true asynchronous host to device copies (e.g., tensor.cuda(non_blocking=True) on transfer streams), avoiding global synchronization (Section 3.2).

To address Pro2, FABLE replaces the conventional buffered filesystem I/O path with a direct-I/O based filesystem-bypass I/O path that eliminates page-cache involvement and redundant copies. FABLE instead accesses the NVMe device via its raw block-device node and opens it with O_DIRECT, which Linux defines as "try to minimize cache effects" by performing I/O directly to/from user-space buffers. Because direct I/O imposes strict alignment constraints, FABLE encapsulates all such details inside the Raw Block Device Manager, which issues aligned positional reads/writes and stages each object into block-aligned, page-locked (pinned) host buffers. Using pinned staging buffers satisfies CUDA's requirement that host buffers must be pinned/page-locked for host to device copies to be truly asynchronous (otherwise *cudaMemcpyAsync* may revert to synchronous behavior and lose overlap). Together, bypassing the page cache and minimizing copies reduces CPU overhead, yielding a lower-jitter SSD to pinned-DRAM to GPU memory path and reducing GPU stall time while data is in transit (Section 3.3).

3.2. Fine-Grained Prefetching

When the inference runtime schedules at whole-layer granularity over an SSD to DRAM to GPU memory hierarchy, the GPU often waits for data. A layer can launch only after its weights—and, for attention, the KV blocks needed by that layer—are available in GPU memory. As a result, the uncovered portion of SSD to DRAM to GPU memory staging tends to concentrate at layer boundaries, appearing as GPU stalls (idle time) and increasing end-to-end inference latency.

To mitigate GPU stall during inference, FABLE designs the execution architecture in three dimensions. First, FABLE decomposes each layer ℓ into two fine-grained scheduling units, MHA_ℓ and FFN_ℓ, and schedules them as the basic compute units (Section 3.2.1), enabling overlap at sublayer granularity. While the GPU executes MHA_ℓ, FABLE prefetches the FFN_ℓ object into GPU memory. While the GPU executes FFN_ℓ, FABLE stages the next layer's $\mathrm{MHA}_{\ell+1}$ object which include its weights together with the KV blocks required by that attention—through the SSD to DRAM to GPU memory path, increasing compute–I/O overlap under a three-tier hierarchy. Second, the Weight & KV Cache Controller maintains a sliding-window working set and applies a window-based prefetch/evict policy over objects (Section 3.2.2), controlling residency across GPU memory, host DRAM, and SSD to reduce stalls from missing data. Finally, the Stream & Event Manager employs CUDA streams and event-based dependencies to build an asynchronous execution path (Section 3.2.3). With page-locked host buffers and asynchronous CUDA copies, these streams allow host to device transfers to overlap with kernel execution, further reducing GPU waiting time.

3.2.1. Fine-Grained Execution task with data

FABLE uses three storage tiers: NVMe SSD, host DRAM, and GPU memory. The weights and the KV blocks needed by attention are staged through SSD to DRAM to GPU memory hierarchy rather than residing permanently in GPU memory. Data movement can therefore become a major cost in inference. If the inference runtime schedules at whole-layer granularity, it frequently waits at layer boundaries: a layer can begin only after its required weights and KV blocks have been staged into GPU memory. In this case, waiting I/O latency manifests as GPU stall and increases end-to-end inference latency. As discussed in the background (Section 2.1), weight size grows rapidly with model scale, while the KV cache grows with both model scale and input prompt length, further amplifying staging cost.

By inspecting the Transformer structure, we observe that MHA phase and FFN phase are independent in terms of weights: FFN phase depends on the intermediate activation produced by MHA phase, but it does not require the MHA phase weights to remain resident once MHA phase finishes. Moreover, each submodule's weights consist of multiple matrix tensors which whole-layer staging unnecessarily coarse. Figure 5 illustrates the consequence of exploiting this coarse structure: the system first stages all inputs for layer ℓ—the weights for MHA phase and FFN phase, together with the KV blocks required by MHA phase—from SSD/DRAM into GPU memory, and only after this "$Loading_\ell$" phase completes does the "$Computation_\ell$" phase begin. As a result, I/O latency appears as a large idle gap before computation.

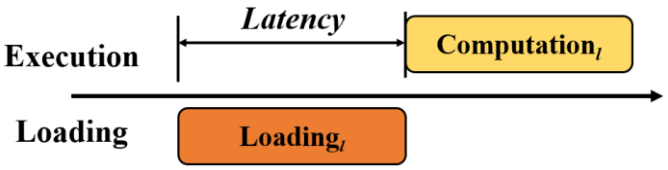


Figure 5. Whole Layer Inference

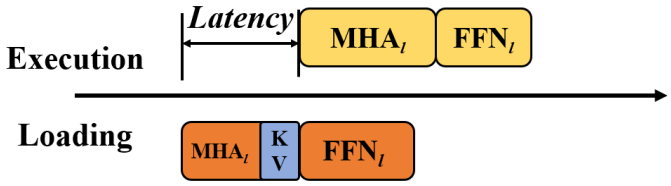


Figure 6. Fine-grained Prefetching Inference

FABLE enables fine-grained prefetching at sublayer granularity, as shown in Figure 6. We model each transformer layer's $Computation_\ell$ as two execution units and treat them as independently schedulable work items. Concretely, FABLE treats each layer ℓ as two schedulable units, MHA_ℓ and FFN_ℓ with these unit needed data is MHA_ℓ object and FFN_ℓ object to replace $Loading_\ell$ in Figure 6. Notably, an MHA_l object that contains the weights needed by the MHA_ℓ unit and the layer-ℓ KV cache, as well as an FFN_ℓ object that contains the weights needed by the FFN_l unit. To maximize overlap, FABLE separates computation and data movement onto different CUDA streams: MHA_ℓ and FFN_ℓ kernels are launched on compute streams, while SSD to DRAM reads and DRAM to GPU memory copies are issued on transfer streams; KV reads and KV write-backs use separate transfer streams to avoid mutual interference. This creates a steady-state two-step alternation that eliminates the whole-layer "ready" barrier and increases compute–I/O overlap. In Figure 6 before MHA_ℓ executes, FABLE stages the inputs for the next attention, i.e., the MHA_ℓ object. While MHA_ℓ executes, FABLE stages FFN_ℓ object. FABLE checks readiness for each unit separately. FFN_ℓ is ready once its weights and input activation are resident in GPU memory that is FFN_ℓ object, and MHA_ℓ is ready once its weights, input activation, and required KV blocks are resident that is MHA_ℓ object. The inference runtime can therefore launch MHA_ℓ or FFN_ℓ as soon as that sublayer's own inputs become available, instead of waiting for the whole layer to finish loading which reduces the "whole-layer-ready" barrier and substantially increases compute–I/O overlap.

The effectiveness of sublayer-granularity overlap depends on whether each sublayer provides enough compute slack to cover the SSD to DRAM to GPU memory staging of the next sublayer's object. With short prompts and low request concurrency, MHA_ℓ and FFN_ℓ kernels are brief, leaving limited slack. The uncovered portion of staging therefore remains on the path as transfer-induced stalls. Our target workload is long-context, high-throughput serving on consumer-grade GPUs: prompts of around 2K tokens together with high concurrency (e.g., batch sizes of 32–64) make each $\text{MHA}_\ell/\text{FFN}_\ell$ step more compute-intensive, so staging for the next unit often fits within the current unit's compute slack. This pipeline overlaps a large fraction of object movement with computation. Even when the compute slack is insufficient to fully hide a transfer, sublayer-granularity scheduling fragments what would have been a single large layer-boundary I/O gap into smaller, per-sublayer bubbles, reducing transfer-induced GPU stall and lowering end-to-end inference latency compared with whole-layer system. This benefit manifests differently in prefilling and decoding, where the compute–I/O balance changes.

Viewed across prefilling and decoding, FABLE applies the same sublayer-granularity pipeline, where data transfers are scheduled ahead of use and overlapped with ongoing computation to reduce GPU stalls, but the dominant bottleneck differs. During prefilling, each forward pass processes the full prompt and thus tends to be more compute-intensive, providing larger compute slacks to overlap staging of upcoming $\text{MHA}_\ell/\text{FFN}_\ell$ objects. During decoding, computation per step is small because each iteration generates only one token, yet it still requires the weight and a read of the accumulated KV cache. This leaves little compute slack to hide transfers, so any residual SSD/DRAM-to-GPU movement is more likely to appear as an exposed stall. As a result, decoding is precisely where FABLE's fine-grained scheduling and prefetching matter most, because they create more overlap opportunities within each layer and reduce GPU waiting time. Accordingly, we evaluate FABLE on both time-to-first-token (TTFT) and steady-state per-token throughput, and show that sublayer-granularity scheduling reduces transfer-induced stalls and improves decode throughput under our target workloads.

To realize this overlap in a memory-constrained setting, FABLE decomposes the inference runtime into two components. The Weight & KV Cache Controller maintains the sliding-window working set and makes sublayer-granularity residency decisions for $\text{MHA}_\ell/\text{FFN}_\ell$ object, emitting concrete staging and eviction plans across SSD, host DRAM, and GPU memory. The Stream & Event Manager executes these plans by issuing asynchronous transfers and kernel launches on multiple CUDA streams with event-based dependencies, implementing the pipeline. We detail the controller and the stream/event machinery in Section 3.2.2 and Section 3.2.3, respectively.

3.2.2. Weight & KV Controller

On consumer-grade machines, GPU memory capacity is limited, and host DRAM is often insufficient to hold full-precision weights together with the KV cache required by long prompts and high concurrency, making NVMe SSD the only tier that can reliably store the complete model state. As discussed in Section 2.3, the substantial bandwidth and latency disparities across GPU memory, DRAM, and SSD impose a tiered resource hierarchy. GPU memory offers the highest bandwidth but the smallest capacity. DRAM provides greater capacity but is bounded by PCIe transfer bandwidth and latency. SSD provides the largest capacity while incurring the highest access latency.

The Weight & KV Cache Controller in Section 3.2.2 is FABLE's control plane for hierarchical storage. Section 3.2.2.1 (Hierarchical Weight and KV Cache Placement) defines the controller-managed objects (for MHA_ℓ and FFN_ℓ) and their staging states across GPU memory, pinned DRAM, and NVMe SSD. Building on this placement state, Section 3.2.2.2 (Window-based Prefetching and Eviction) specifies the controller policy: a sliding-window drives cross-tier staging actions, including prefetch/offload and promote/demote path for the upcoming unit objects. The controller combines the placement state and window policy to continuously update tier residency and budgets, and emits dependency-annotated movement actions that are materialized by the Raw Block

Device Manager on the SSD path and by the Stream & Event Manager on the GPU path.

3.2.2.1. *Hierarchical Weight & KV Cache Placement*

FABLE organizes model state across a three-tier hierarchy comprising GPU memory, pinned host DRAM, and NVMe SSD. This subsection focuses on where weights and KV blocks reside at each tier and how move along the SSD to DRAM to GPU memory path under the control of the Weight & KV Controller.

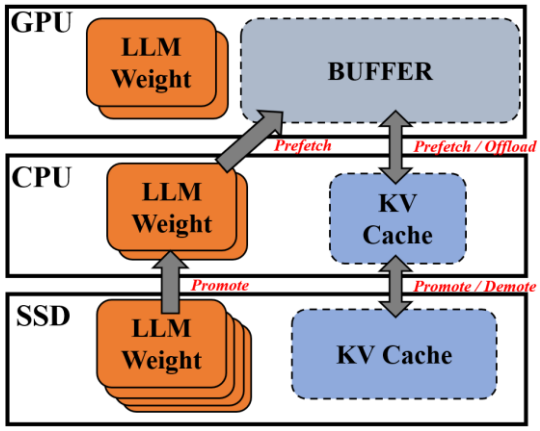


Figure 7. FABLE Workflow

FABLE architecture Figure 7 incorporates a three-tier pipeline driven by a sliding-window. The controller stages upcoming execution unit object across tiers ahead of use. It first promotes required data from SSD into pinned DRAM, and then prefetchs it from pinned DRAM into GPU memory, so that MHA_ℓ/FFN_ℓcan launch as soon as their MHA_ℓ/FFN_ℓ objects are ready. For the KV cache, FABLE offloads KV blocks immediately after MHA_ℓ finishes and then demotes them from DRAM, which frees DRAM space for subsequent requests. Because the window can extend beyond a single layer when budgets permit which set a larger GPU buffer to store more objects, FABLE can maintain a larger lookahead distance than layer-granular, one-step-ahead overlap. Together with sublayer-granularity scheduling and asynchronous stream/event execution, these choices better hide cross-tier latency.

On the weights side, FABLE explicitly distinguishes large weights from small weights, streaming only the former while keeping the latter resident on the GPU. Large weights are the per-layer matrices used by MHA_ℓ and FFN_ℓ, including the attention projections $\{W_Q^\ell, W_K^\ell, W_V^\ell, W_O^\ell\}$and the FFN matrices $\{W_1^\ell, W_2^\ell, W_3^\ell\}$. These matrices dominate the model footprint and are therefore the primary target of streaming. In contrast, small weights—such as per-layer normalization parameters (e.g., LayerNorm/RMSNorm) and other small per-token constants—are typically kilobytes per layer and occupy only a small fixed budget even across many layers. FABLE pins these small weights in GPU memory to avoid repeatedly staging them along the SSD to DRAM to GPU path at every step. This eliminates many tiny transfers and keeps the pipeline focused on the large matrices that dominate bandwidth and latency.

For KV cache organization, FABLE follows the block-based (paged) KV-cache design commonly used in serving systems such as vLLM and SGLang: the per-layer KV cache is partitioned into fixed-size KV blocks, where each block stores the keys and values for a consecutive span of tokens of a single request at a single layer (512 tokens in our implementation). FABLE places these KV blocks across GPU memory, host DRAM, and NVMe SSD, with SSD as the persistent backing store. After KV blocks are generated on the GPU, FABLE immediately offloads them to host DRAM. Host DRAM is used primarily as a staging tier rather than a long-term cache. DRAM staged KV blocks are further demoted to SSD immediately. When MHA_ℓ is scheduled, FABLE prefetches the KV blocks required by that attention together with the MHA_ℓ object.

3.2.2.2. *Window-based Prefetching and Eviction*

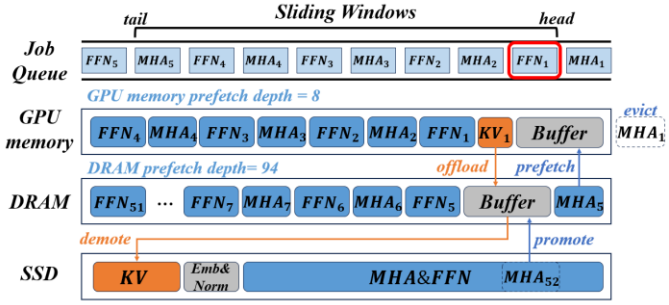


Figure 8. Window-based Weight & KV Cache Prefetching and Eviction

In offloading-based LLM inference, GPU memory cannot hold all model weights and the per-request KV cache simultaneously, so they are stored in host DRAM or SSD and staged to the GPU during execution. Representative systems such as FlexGen prefetch weights at layer granularity with a fixed one-layer prefetch depth, loading the weights of layer $\ell+1$ while computing layer ℓ. With only a one-layer look-ahead, any SSD to DRAM to GPU transfer tail that cannot be hidden by the current layer's compute slack surfaces as GPU stalls while waiting for required weights or KV inputs. FABLE therefore adopts a sliding-window controller that tracks a near-future working set and orchestrates cross-tier staging actions—promote/demote and prefetch/offload—overlapping transfers with computation whenever possible. In our implementation, each window is explicitly bounded by tier budgets. The GPU-side window is capped by D_{GPU} and the DRAM-side window is capped by D_{DRAM}. These equations serve to translate fixed tier budgets into a concrete look-ahead length, showing that the prefetch depth is capacity-limited rather than a free tuning knob. As a result, the controller can maintain the largest possible near-future working set that fits in each tier, and the window advance naturally determines when objects must be evicted, offloaded, or demoted to respect the budgets. In these equations, B_{GPU} and B_{DRAM} denote the byte budgets reserved for in-window objects in GPU memory and pinned DRAM, respectively, and S_{GPU} and S_{DRAM} denote the typical per-object footprint in each tier in bytes.

$$D_{GPU} = (B_{GPU} / S_{GPU}) \quad (1)$$

$$D_{DRAM} = (B_{DRAM} / S_{DRAM}) \quad (2)$$

$$D = \min(D_{GPU}, D_{DRAM}) \quad (3)$$

The Job Queue at the top shows the execution unit order at sublayer granularity in Figure 8. The rightmost position is the head and the leftmost position is tail. The bracket labeled

Sliding Windows highlights a look-ahead region that covers upcoming items in the Unit Queue. The left panel shows where data currently resides across GPU memory / DRAM / SSD by fine-grained object. In this snapshot the inference runtime is executing FFN_1 in the queue head. Meanwhile KV_1 is offloading from GPU memory to DRAM and then demote soon because $\mathrm{MHA}_1$is completed and evicted from sliding window, which is cleared by GPU memory. Caused by window moving, MHA_5 moving on window, so that MHA_5 object is one of the GPU MAX object set. MHA_5 object is prefetched from DRAM to GPU memory. Moreover, CPU MAX object removes MHA_5 object and also evict it in DRAM, and then promote MHA_{52} object because it moving on CPU MAX object set.

Scheduling advances together with the head in the Layer Queue. As soon as a work unit (e.g., MHA_ℓor FFN_ℓ) enters the Sliding Windows region, the controller performs unit-level staging: it prepares the item's required object in the next faster tier before execution reaches it. A farther-ahead object is promoted from SSD to DRAM so that DRAM stays ahead of the queue, and a nearer-term object is prefetched from DRAM to GPU memory so that it becomes immediately usable. Each window advance triggers transfer only for items that have just entered the window, avoided redundant movement and aligning weight staging with the Unit Queue order. KV movement is treated as part of the MHA_ℓ object. For each upcoming MHA_ℓ inside the window, the controller stages the MHA_ℓ object through the SSD to DRAM to GPU memory pipeline. After the MHA_ℓ finishes, the newly generated/updated KV cache is promptly offloaded from GPU memory into a pinned DRAM buffer and then demoted to SSD.

Eviction reclaims upper-tier capacity as the sliding window advances and tier budgets are reached. In FABLE, eviction is performed at object granularity and is aligned with the sliding window: once an object moves out of the window which means all of its dependent GPU work has completed, its upper-tier footprint is reclaimed. Objects may contain immutable parameters only or include online-generated state. For parameter-only objects, eviction is a pure cache drop: FABLE releases their GPU-resident copies without any write-back, since the authoritative weights remain in lower tiers. For MHA_ℓ objects, correctness requires preserving the KV component across decoding. Therefore, when evicting an MHA_ℓ object, FABLE first drains the object's KV component from GPU memory into a DRAM staging buffer and demotes it to SSD immediately, and then releases the corresponding upper-tier space for newly entering objects. In this design, GPU memory and host DRAM serve as bounded cache/staging tiers for in-window objects, while SSD remains the backing store.

The overlap implied is implemented by running computation and data movement on separate CUDA streams and coordinating them with CUDA events. CUDA streams act as ordered work queues, and events provide fine-grained markers that other streams can wait on to enforce "ready-before-use" dependencies without globally blocking the device.

3.2.3. Stream & Event Manager

The Stream & Event Manager is FABLE's GPU-side execution substrate. It takes the controller's sublayer granularity actions with dependencies and turns them into a non-blocking plan of asynchronous data transfers and GPU kernel launches on a fixed set of CUDA streams. It stores transfers on dedicated transfer streams and kernels on compute streams, records an event after each transfer, and makes the consuming compute stream wait on that event before launching the dependent kernels. CUDA streams serve as ordered work queues, and CUDA events express cross-stream "ready-before-use" dependencies without host-side blocking or global device synchronization. For host–device transfers, asynchronous execution and overlap with kernels require streams and page-locked (pinned) host buffers (e.g., *non_blocking*=True is effective when the source is pinned).

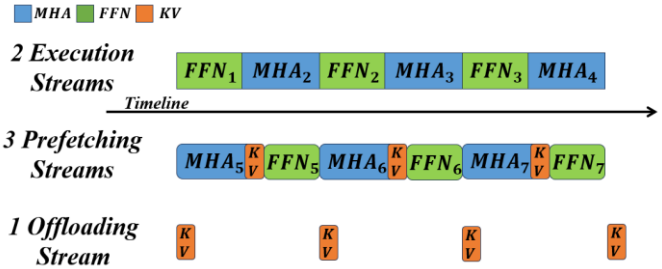


Figure 9. Multiple Streams Workflow

As illustrated in Figure 9, the Stream & Event Manager uses a fixed six-stream layout to pipeline execution, prefetching, and offloading. Concretely, we use six CUDA streams: two execution streams (MHA, FFN), two weight prefetching streams (MHA-weight, FFN-weight), one KV prefetching stream, and one KV offloading stream. We distinguish two concepts throughout this section: a unit is a schedulable GPU compute task (MHA_ℓ / FFN_ℓ), while an object is the corresponding input bundle that must be materialized before the unit can run. In particular, an MHA_ℓ object includes both the MHA_ℓ weights and the per-request KV blocks needed by attention at layer ℓ, whereas an FFN_ℓ object contains only FFN_ℓ weights. For readability, Figure 9 visualizes the schedule as three logical lanes. On the execution lane, two compute streams launch GPU kernels (e.g., $\mathrm{MHA}_1\ to\ \mathrm{FFN}_1\ to\ \mathrm{MHA}_2\ to\ \cdots$): one stream is dedicated to MHA_ℓ kernels and the other to FFN_ℓ kernels. Ahead of execution, two weight prefetching streams enqueue asynchronous DRAM to GPU copies for the weights of upcoming $\mathrm{MHA}_\ell$and FFN_ℓ and record a weight-ready event for each (e.g., execution is at layer 0 while staging has advanced to around layer 6 in the figure). The corresponding compute stream waits on that event before issuing the dependent unit. The KV-prefetching stream brings the KV blocks required by upcoming MHA_ℓ into GPU memory and records a KV-ready event. Although KV cache is part of the MHA_ℓ object, FABLE still isolates KV traffic onto dedicated transfer streams because the KV portion has different transfer volume, timing, and write-back behavior than weights. The last stream is offloading stream containing KV offloading stream only. The KV-offload stream asynchronously drains KV blocks from GPU memory back to pinned host memory after it become safe to release (e.g., after the last dependent attention completes), so that background write-back does not serialize with weight staging on the inference path. Overall, for each MHA_ℓ object, the Stream & Event Manager materializes a small dependency pattern—transfer streams record "ready" events, and compute

streams wait on them—using CUDA event record/wait (e.g., *cudaEventRecord / cudaStreamWaitEvent*), which coordinates transfers and kernels on-device without CPU-side global synchronization. CUDA streams are ordered queues of GPU operations, and cross-stream event waits are the standard mechanism to express such dependencies. All host and device transfers use page-locked (pinned) host buffers. CUDA requires host buffers to be pinned for copies involving CPU memory to execute asynchronously. Otherwise, *cudaMemcpyAsync* may revert to synchronous behavior and lose overlap with other work.

FABLE uses two logical priority classes to protect the latency-critical portion of the decoding pipeline. We assign high priority to the MHA execution stream because MHA units lie on the per-layer path and directly gate next-token generation. At the controller level, we also treat the prefetching of MHA_ℓ object as high priority: both the MHA-weight prefetching stream and the KV prefetching stream produce the readiness conditions for the next MHA_ℓ unit, and a miss on either component manifests as a hard GPU "data-not-ready" stall. In contrast, we assign normal priority to the FFN compute stream and FFN weight prefetching stream because FFN_ℓ units are downstream of MHA_ℓ unit and often have more slack. Likewise, KV offloading is best-effort and can be delayed as long as it does not threaten capacity. CUDA stream priority is primarily a scheduling hint and does not guarantee a specific execution order or preempt already running work. Moreover, in the current implementation stream priority affects compute kernels but has no effect on DRAM to GPU memory operations. Therefore, when PCIe bandwidth or copy engines become contended, FABLE enforces transfer "priority" via admission control in the controller: it issues high-priority staging actions earlier, caps the number of in-flight low-priority transfers, and defers best-effort FFN prefetching and KV offloading whenever doing so would risk delaying MHA_ℓ object readiness.

The Stream & Event Manager encodes both data readiness and MHA to FFN control dependencies into GPU work queues via event-driven cross-stream synchronization. This avoids CPU-side blocking waits. FABLE uses PyTorch CUDA primitives to realize streams and events. It instantiates a fixed set of six persistent streams with *torch.cuda.Stream*(device=..., priority=...): two compute streams (MHA, FFN) and four transfer streams (MHA-weight H2D, FFN-weight H2D, KV H2D, KV D2H). We use two logical priority classes (*PRIO_HIGH=-1 and PRIO_NORM=0*) and map them through a small helper (e.g., *_safe_priority*) so that the requested priorities remain valid on the target stream. FABLE stages host-side tensors in pinned memory to enable asynchronous transfers. In practice, we obtain pinned buffers via *tensor.pin_memory()* and a custom aligned pinned allocator (e.g., *alloc_pinned_aligned*). We then enqueue bidirectional transfers between host and device copies on the designated transfer streams with *non_blocking=True*, so transfers can overlap with kernel execution when the copy engines and PCIe bandwidth permit.

Dependencies are expressed with reusable CUDA events exposed by PyTorch. The manager allocates events via *torch.cuda.Event*(enable_timing=False, blocking=False) and reuses them through an event pool to reduce overhead. A producer publishes completion by calling *event.record*(stream) on the producing stream. A consumer enforces ready-before-use by calling *stream.wait_event*(event) on the consuming stream. We apply the same mechanism to capture sublayer control dependencies: the MHA compute stream records an MHA-finished event, and the FFN compute stream waits on both the MHA-finished event and the FFN-weights-ready event before launch. As the sliding window advances, transfer streams prefetch upcoming objects while compute streams execute the current sublayer, and event chaining sustains compute–I/O overlap in steady state.

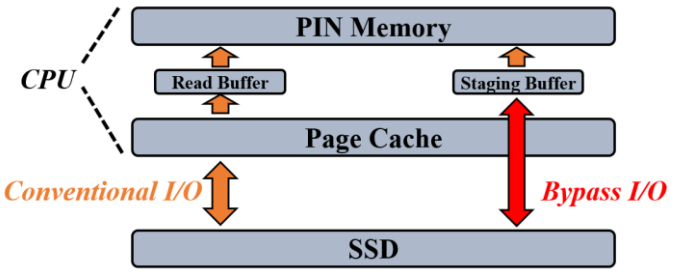
Overall, the Stream & Event Manager operationalizes sublayer-granularity scheduling and the sliding-window policy by mapping controller actions onto a fixed set of CUDA streams and event mechanism. It lowers upcoming MHA_ℓ/ FFN_ℓ execution into non-blocking sequences of asynchronous copies and kernel launches, and expresses dependencies through record-event/wait-event pairs so that readiness is enforced without the host repeatedly alternating between blocking waits. In addition, FABLE uses two logical priority classes to protect the latency-critical portion of decoding: it assigns higher stream priority to MHA_ℓcompute kernels (which gate next-token generation), while treating FFN_ℓcompute and background work as best-effort. Because CUDA stream priority is primarily a kernel-scheduling hint (and does not provide a guaranteed priority mechanism for host–device copies under PCIe/copy-engine contention), FABLE enforces transfer "priority" via admission control in the Weight & KV Controller, issuing critical-path staging actions earlier and deferring low-priority prefetching/offloading when doing so would risk delaying MHA_ℓ readiness.

FABLE stores model weights on SSD as a single contiguous binary file (the weight image) together with a compact lookup table for offset-based access. This contrasts with Safetensors, which stores tensor metadata in a JSON header and locates each tensor by byte ranges/offsets. In practice, checkpoints may also be sharded, with an index that maps parameter names to multiple shard files. FABLE instead packs tensors in the exact consumption order of the $\mathrm{MHA}_\ell/\mathrm{FFN}_\ell$pipeline and records a compact per-sublayer, e.g., (offset, length) ranges for each MHA_ℓ and FFN_ℓ weight object) plus preallocated, aligned regions for KV blocks. This turns SSD reads into simple positional reads and KV persistence into positional writes over preassigned extents. The Raw Block Device Manager materializes these bidirectional transfers between SSD and DRAM I/O, and the Stream & Event Manager overlaps the subsequent bidirectional transfers between DRAM and GPU memory copies with execution using the same event-driven readiness mechanism. In Section 3.3, we show that this layout enables a raw/direct-I/O backend that can bypass the page cache when alignment constraints are satisfied.

3.3. Filesystem Bypass

On consumer-grade hosts, model state often spills to NVMe SSD, making the SSD to host DRAM read path a first-order bottleneck under high concurrency. With conventional I/O, reads go through the kernel page cache, which can add extra copies and management overhead for our streaming, low-reuse access pattern. FABLE therefore uses a filesystem-bypass path based on direct I/O to bypass the OS page cache on the read path.

3.3.1. *Raw Block Device Manager*



(a) Conventional I/O (b) Bypass I/O

Figure 10. Conventional I/O compared with Bypass I/O

As illustrated in 錯誤！找不到參照來源。, under the Conventional I/O path, SSD data is first brought into the OS page cache and then copied into a user read buffer when the next step is host-to-device transfer, the data must reside in pinned memory, so the runtime stages the read buffer into pinned buffer, introducing an additional copy. Direct I/O is attractive in this setting because it avoids the extra copy from the cache to the user buffer and reduces page-cache involvement. Beyond extra copies, the conventional path also incurs filesystem and page-cache bookkeeping on the read path, and SSD to DRAM traffic is interleaved with page-cache management activities such as page allocation or eviction and writeback, which can introduce bandwidth variability and tail-latency jitter under long-context and highly concurrent inference. In addition, large-model checkpoints are commonly sharded across multiple weight files, which can lead to many files read during weight staging. To reduce per-file overhead and make weight staging largely sequential, FABLE packs model weights into a single contiguously laid-out object store; we apply the same packing to FS baselines to avoid conflating layout with the I/O path. As illustrated in 錯誤! 找不到參照來源。, under the Bypass I/O path, FABLE accesses the SSD via its raw block-device node (e.g., /dev/nvme0n1) and requests direct I/O by opening it with *O_DIRECT*, which Linux defines as "try to minimize cache effects" by performing I/O directly to user-space buffers. The Linux manual states that *O_DIRECT* may impose alignment restrictions on the I/O offset, the I/O length, and the user-space buffer address, and misaligned *O_DIRECT* I/O may either fail with EINVAL or fall back to conventional I/O, depending on the filesystem and kernel. Accordingly, the Raw Block Device Manager opens the block device with *O_DIRECT* | *O_LARGEFILE* and queries the device logical block size once via *ioctl*(BLKSSZGET), which Linux documents to obtain the block device logical block size. Because our bypass path targets a raw block device, we use *BLKSSZGET* to obtain the device logical block size as the alignment quantum. *STATX_DIOALIGN* is applicable when using file based direct I/O. If the query fails, we conservatively fall back to 4096 B (a common page/sector size and a multiple of 512 B), and we validate alignment assumptions at startup. In our evaluation setup, *BLKSSZGET* returns a 512-byte logical block size, and we align all direct I/O requests to this granularity. If validation fails, we abort rather than silently issuing misaligned direct I/O. By construction, every request we issue is aligned; we additionally validate these constraints in user space and fail fast on any violation, preventing ambiguity between EINVAL failures and silent conventional I/O fallbacks. These checks prevent unintended page-cache involvement and yield a low-jitter SSD to pinned-DRAM staging path that integrates cleanly with asynchronous GPU copies. In summary, filesystem bypass benefits FABLE by reducing host-side copies on the data path, reducing filesystem/page-cache overhead on the control path, and enabling largely sequential weight staging via offline packing.

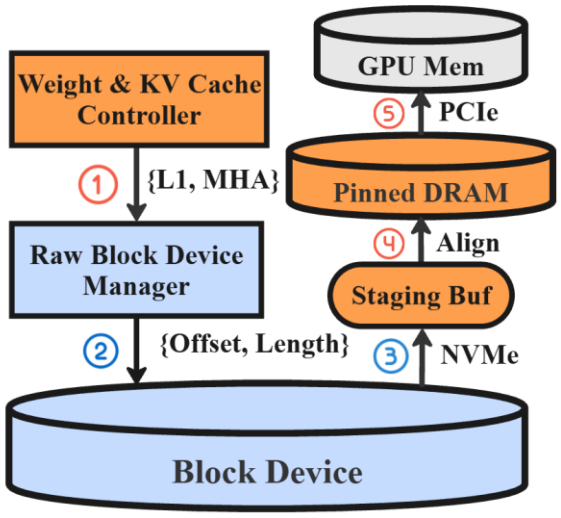


Figure 11. I/O Workflow

As illustrated in Figure 11, the filesystem-bypass pipeline consists of five pipeline stages that translate a logical object-read request into an aligned NVMe read and an asynchronous H2D copy. 1) Request formation. When the Weight & KV Cache Controller determines that an upcoming compute unit will soon need data(MHA_ℓ/FFN_ℓ object), it issues a logical read request that identifies the target object, and the destination pinned host tensor to be filled. 2) Layout-based translation and alignment. Instead of consulting a per-layer lookup table for weights, FABLE derives on-device byte ranges from layout parameters recorded at startup (e.g., a base offset and fixed, padded object sizes for MHA_ℓ/FFN_ℓ). The Raw Block Device Manager translates the logical request into one or more (offset, length) extents and rounds them to the alignment required by direct I/O. 3) SSD-to-staging fetch. The Raw Block Device Manager issues positional direct-I/O reads to fetch the aligned extents from the block device into a block-aligned staging buffer. 4) Alignment and slicing. If the destination pinned tensor already satisfies the direct-I/O alignment and size requirements, the manager reads directly into that destination buffer. Otherwise, it reads an aligned superset into the staging buffer and then copies the requested subrange into the destination pinned tensor used for the subsequent host to device transfer. 5) Host-to-GPU transfer and readiness. The Stream & Event Manager enqueues an asynchronous H2D copy from pinned host memory to GPU memory and records a data-ready event on the transfer stream. The consuming compute stream waits on that event before launching the dependent MHA_ℓ/FFN_ℓ kernels, enabling overlap without CPU-side blocking waits. The five stages together ensure that every object read is

block-aligned for direct I/O and becomes GPU-ready through asynchronous transfer, minimizing software overhead and reducing exposed data-wait stalls.

Direct I/O comes with an important caveat: Linux may impose alignment restrictions on the I/O offset, the I/O length, and the user-space buffer address. These restrictions vary by filesystem and kernel version, and misaligned requests may either fail with EINVAL or fall back to conventional I/O. This caveat matters in LLM serving because checkpoint formats describe tensors as byte ranges (offset, length) that are not naturally aligned to the device block size. FABLE therefore packs tensors into a block-aligned image, while the Raw Block Device Manager still provides a staging-and-slice path to safely serve any unaligned/sliced logical reads. To make direct I/O usable without requiring upper layers to be aware of block-alignment constraints (even though logical requests are expressed as byte ranges), the manager introduces long-lived, block-aligned staging buffers in host memory. It first derives a conservative alignment quantum from the device logical block size and then enforces a "three-alignment" rule for all device I/O: offsets, lengths, and buffer base addresses are multiples of this quantum. For already aligned requests, data flows directly from SSD into either a caller-provided aligned pinned buffer or an internal aligned staging buffer. For small reads with arbitrary byte offsets/lengths, the manager rounds the request to an aligned extent (rounding the start down to the nearest block boundary and the end up accordingly), performs a single aligned bulk read into a staging buffer, and then copies out the requested subrange to the upper layer. Finally, the manager validates the three-alignment rule in user space and fails fast on any violation, preventing unintended conventional I/O fallback and ensuring that the SSD to pinned-DRAM staging path consistently satisfies *O_DIRECT* alignment constraints.

FABLE couples a sliding-window policy with a unified, block-level I/O interface for all streamed objects. Concretely, the controller materializes near-future demand as a priority-ordered task queue that promotes/prefetches and demotes items between NVMe SSD and pinned host DRAM; advancing the window simply changes which (offset, length) byte ranges enter or leave this queue.

FABLE moves weights and KV blocks between NVMe SSD and pinned host DRAM using offset-based, block-level reads and writes. For weights, SSD reads are mostly large and contiguous because the on-disk layout places each MHA_l and FFN_l weight region sequentially. FABLE therefore uses a single weight-promotion worker that processes promote tasks in sliding-window order and reuses a pinned, block-aligned staging buffers. In contrast, KV access is smaller and more fragmented, so FABLE uses a KV-prefetch thread pool and keeps more requests in flight. Each worker issues positional reads via *pread*(), which avoid synchronization on a shared file offset. Maintaining multiple outstanding requests also improves device-level parallelism, since NVMe supports many in-flight commands.

To sustain this pipeline under fixed DRAM and GPU budgets, FABLE also needs a demote path to evict cold KV blocks from host memory**.** On the demote path, FABLE splits KV demotion into two stages. A packer batches cold KV blocks into block-aligned write buffers, and a writer thread drains these buffers to SSD using offset-based *pwrite*() calls. Under sudden memory pressure, FABLE can spawn an extra spill worker to accelerate demotion. Because KV blocks are a performance cache rather than correctness state, the demote path does not require durability (e.g., no fsync per block).

Each queued item is described by one or more on-device byte ranges (offset,length) derived from the packed layout and a runtime manifest; although weights and KV blocks share the same offset-based interface, they exhibit different access characteristics, so FABLE uses different workers to execute the same task queue. Section 3.3.2 describes how these byte ranges are constructed.

3.3.2. *On-Disk Layout*

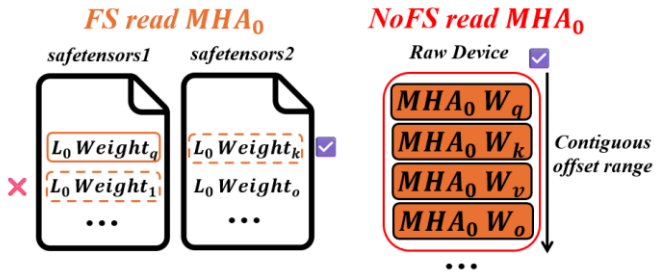


Figure 12. FS Sharded-File Reads vs. NoFS Contiguous-Range Reads

To make filesystem-bypass I/O effective at runtime, FABLE aligns on-disk placement with the same minimal units used by the sublayer-granularity pipeline. In the offline phase, FABLE rewrites the model checkpoint into a reserved raw NVMe region as a contiguous weight image whose physical order matches FABLE's consumption order. Since large-model checkpoints are often sharded across multiple files, consolidating them into one packed image avoids many separate file reads during weight staging and makes accesses largely sequential. For fair comparisons, we apply the same packed layout to FS baselines to avoid conflating layout effects with storage-path overhead. Figure 12 illustrates this motivation using MHA_0 as an example. Under the original sharded Safetensors layout, the tensors needed by MHA_0(e.g., W_q and W_k) can reside in different shard files, so staging MHA_0may require multiple file reads. In this setting, filesystem lookahead typically reads ahead contiguous bytes within the currently accessed shard into the page cache, so it may fetch adjacent data (e.g., the depicted $Weight_1$) that is not required by the current MHA_0read, wasting bandwidth and cache space. Repacking into the NoFS raw-device image makes the required tensors (W_q, W_k, W_v, W_o) occupy a single contiguous device-offset range, so fetching MHA_0becomes a sequential read over that range, and lookahead or runtime prefetch brings in upcoming required weights rather than unrelated bytes. In common sharded checkpoints, an index file maps each tensor name to a shard file. The checkpoint is typically stored in Safetensors, which contains a small header (an 8-byte header-length field plus a JSON header with per-tensor dtype, shape, and data offsets) followed by the raw tensor body. The packer preserves each tensor's payload bytes verbatim and only changes their placement: it reorders and concatenates tensors in FABLE's sublayer-major consumption order, inserting zero padding only to satisfy aligned direct-I/O reads. The packer runs on the target host and uses the same alignment quantum as

the runtime, derived from the device logical block size, so object boundaries are block-aligned by construction. As a result, runtime alignment rounding is usually a no-op and is retained mainly for robustness and for merging adjacent reads.

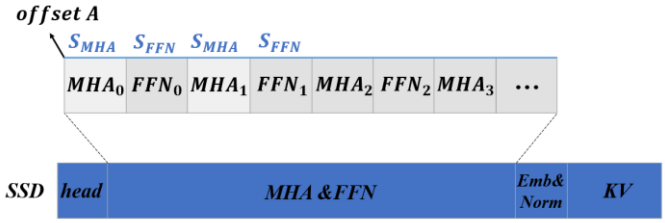


Figure 13. Raw Block Device layout

Figure 13 shows the layout of the reserved raw NVMe region. After a header which stores objects' metadata in SSD, FABLE stores the streamed weights in a single contiguous weight image in strict layer-major order, with MHA_ℓ immediately followed by FFN_ℓ for every layer ℓ. A separate region at the end of the device is reserved for KV blocks. Small always-resident parameters (e.g., per-layer normalization parameters) can be stored in a small region and pinned in GPU memory, so they are not part of the streaming path. A key simplification is that, for the model family we target (homogeneous transformer layers), the packed byte sizes of MHA_ℓ and FFN_ℓ weights are constant across layers once padding is applied. Therefore, weights do not require a per-layer lookup table. The runtime needs only a base offset and two constants:

- A: the base device offset of MHA_0 in the weight image.
- S_{MHA}: the padded byte size of MHA_ℓ weights (same for all ℓ).
- S_{FFN}: the padded byte size of FFN_ℓ weights (same for all ℓ).

Then the device extents are computed by simple arithmetic. The base case $\ell = 0$ follows directly: $off(MHA_0) = A$ and $off(FFN_0) = A + S_{\text{MHA}}$. Although inference executes layers sequentially (so an implementation may also maintain a running cursor that alternates adding S_{MHA} and S_{FFN}), FABLE present the closed-form mapping to make the layout explicit and to support the sliding-window controller, which may stage layers several steps ahead and evict out-of-window layers without consulting any per-layer table.

$$stride = S_{\text{MHA}} + S_{\text{FFN}} \quad (4)$$

$$off(MHA_\ell) = A + \ell \cdot stride \quad (5)$$

$$off(FFN_\ell) = A + \ell \cdot stride + S_{\text{MHA}} \quad (6)$$

In our workflow Figure 14, the packer enumerates tensors and writes their payload bytes in deterministic layer-major, sublayer-major order: for each layer ℓ, it writes the tensors consumed by MHA_ℓ followed immediately by the tensors consumed by FFN_ℓ. Each tensor payload is written byte-for-byte and then padded with zeros as needed so that subsequent I/O can be issued as aligned positional reads/writes. This padding ensures that any aligned direct-I/O fetch of a tensor or object never overlaps neighboring payloads; any extra bytes read due to rounding are padding rather than unrelated tensors, keeping runtime copy/unpack logic simple. Offline packing emits Shapes_Meta (shapes_meta.json), which records per-tensor attributes (name, layer, dtype, shape, nbytes, and canonical packing order), as well as the reserved header size. Shapes_Meta intentionally omits per-layer placement tables to avoid redundant metadata, since all offsets can be reconstructed from the canonical packing order and per-tensor sizes in Shapes_Meta.

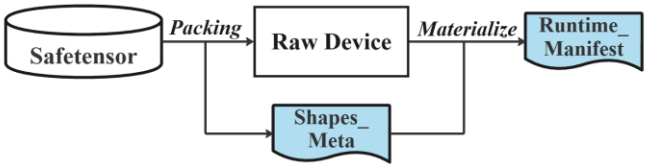


Figure 14. Packing & materialize workflow

At runtime startup, FABLE materializes a lightweight RuntimeManifest (runtime_manifest.json) from Shapes_Meta and the probed device alignment. We query the device logical sector size and derive an alignment/block size used for rounding extents. Since O_DIRECT alignment restrictions vary across filesystems and kernel versions, explicitly recording the probed alignment in the runtime manifest avoids relying on filesystem-dependent behaviors. RuntimeManifest records only a small set of layout parameters—$(A, S_{\text{MHA}}, S_{\text{FFN}}, stride)$—so the runtime can compute the extents of $\text{MHA}_\ell/\text{FFN}_\ell$ weights by the formula above.

KV blocks cannot be addressed by a per-layer formula alone because placement is request-scoped and allocated dynamically. Once a KV block is assigned a slot_id, its device offset is computed by a simple fixed-stride formula. FABLE therefore reserves a dedicated KV region on the raw device and manages it as an array of fixed size, aligned slots. The runtime maintains an in-memory KV directory that maps each logical KV block $(request\_id, \ell, block\_idx)$ to a slot_id. The device offset of a KV slot is computed as $off(slot\_id)$ where $slot\_bytes$ is the padded byte size of one KV block slot and kv_base is the start offset of the KV region. With this scheme, KV prefill/offload becomes simple offset-based reads/writes to preallocated aligned ranges, which the Raw Block Device Manager executes via *pread*()/*pwrite*() (these calls do not change the shared I/O offset and avoid synchronization on a global seek pointer). The subsequent DRAM to GPU staging is then coordinated by the Stream & Event Manager using the same event-based readiness mechanism described in Section 3.2.3.

$$off(slot\_id) = kv\_base + slot\_id \cdot slot\_bytes \quad (7)$$

4. EXPERIMENTAL RESULTS

This section evaluates FABLE as a standalone inference engine on a single consumer GPU. We measure end-to-end inference latency under highly concurrent workloads. We also isolate the contributions of fine-grained prefetching and filesystem-bypass I/O.

4.1. Experimental Setup

Testbeds. All experiments were conducted on the single-node testbed summarized in Table 1, consisting of one NVIDIA GeForce RTX 5080 GPU [32] with 16 GB of VRAM, an Intel Core Ultra 7 265K CPU, 128 GiB of DDR5-4400 DRAM in a 4×32 GiB configuration, and a 2 TB NVMe SSD. Both the GPU and the SSD are attached to the host via PCIe Gen5. The system

runs Ubuntu 24.04 LTS with PyTorch 2.8.0 (cu129) and CUDA 12.9.

Table 1. Experimental testbeds

CPU	Intel Core Ultra 7 265K
GPU	NVIDIA GeForce RTX 5080 16GB
Memory	G Skill DDR5 4400 MHz, 32GiB*4
SSD (Cache Device)	Micron Crucial T700 2TB
OS	Ubuntu 24.04 LTS
Pytorch & CUDA	2.8.0(cu129) & 12.9

This paper implemented FABLE in Python on top of PyTorch [26]. For model execution, FABLE integrates a Transformer-based LLaMA model. FABLE uses multiple CUDA streams and CUDA events to overlap GPU computation with host–device transfers. To ensure these transfers remain asynchronous, data is staged in pinned (page-locked) host buffers. In parallel, FABLE employs a pool of CPU I/O threads to pipeline SSD–DRAM staging of model state, including streamed objects, thereby overlapping inference computation with weight and KV movement.

Table 2. Model & Workload

Model	LLaMA-3.1-70B
Workload	ShareGPT Anthropic/AnthropicInterviewer
Prompt Length	2048 Tokens
Batch Size	1, 16, 32, 64
Generator Length	32 Tokens

Model&Workload. This paper evaluate FABLE using the open-weight Meta Llama 3.1 70B model meta-llama/Llama-3.1-70B and dialogue workloads derived from ShareGPT Anthropic/AnthropicInterviewer [39], with the main settings summarized in Table 2. This paper run inference in BF16 for both weights and activations. We fix the prompt length to 2,048 tokens and the maximum generation length to 32 tokens, and sweep batch sizes of 1, 16, 32, and 64. For batch sizes 32 and 64, prefilling can exceed the device-memory budget due to activation and KV-related workspace requirements, so we enable chunked prefilling and process each prompt in micro-batches of up to 512 tokens. So time-to-first-token include the first decoding step, since chunked prefilling only builds the prompt KV cache and does not emit output tokens. During decoding, FABLE uses topk guided KV prefetching with a fixed prefetch budget equivalent to 512 tokens, prioritizing the most relevant KV blocks for promotion into GPU memory ahead of attention.

To disentangle the contributions of fine-grained prefetching and filesystem bypass, we evaluate FABLE against a set of controlled baselines. FS denotes conventional I/O through the Linux page cache and filesystem which regular files open without O_DIRECT. For the FS variants, we also pack all model weights into a single contiguously laid-out binary file (identical placement policy as NOFS), so the comparison isolates storage-stack overhead rather than confounding effects from random-vs-sequential access/placement. NoFS denotes filesystem bypassing I/O implemented by opening the raw block device. **NoP-FS** is an on-demand offloading baseline, where the runtime fetches weights and KV blocks only when they are needed for computation. **NoP-NoFS** keeps the same on-demand schedule but replaces filesystem-based SSD reads with FABLE filesystem bypass path, isolating the impact of the storage stack. **LaP-FS** is an improved variant of FlexGen[3] and serves as our representative layer-granularity offloading baseline. LaP performs prefetching at layer granularity and overlaps computation with prefetching, but uses the same prefetch depths as FABLE to control for the effect of a larger lookahead window: it increases the GPU-memory prefetch depth from FlexGen's original one-layer setting to 6 layers, and sets the DRAM prefetch depth to 47 layers, matching FABLE. This design allows us to isolate and highlight the benefit of fine-grained prefetching beyond what can be achieved by simply using a larger prefetch depth. **LaP-NoFS** further combines this scheduler with our filesystem-bypass path to quantify the benefit of filesystem bypass under an established layer-granularity design. **FABLE-FS** enables fine-grained prefetching while retaining conventional filesystem path, whereas **FABLE** represents the full system that integrates both fine-grained prefetching overlap and filesystem bypass. We also considered a **DRAM-only** configuration that eliminates SSD and attempts to keep all offloaded state in host DRAM. On FABLE testbed, this configuration cannot complete the target workloads and fails with host-memory out-of-memory errors, since full-precision 70B-scale weights alone require on the order of 140 GB at BF16, not including KV cache and runtime buffers. We therefore omit DRAM-only results from the plots.

4.2. End-to-end Performance

In FABLE end-to-end evaluation we measure end-to-end latency, a phase breakdown of prefilling and decoding, time to first token for each baseline, thereby characterizing FABLE's serving performance under high concurrency.

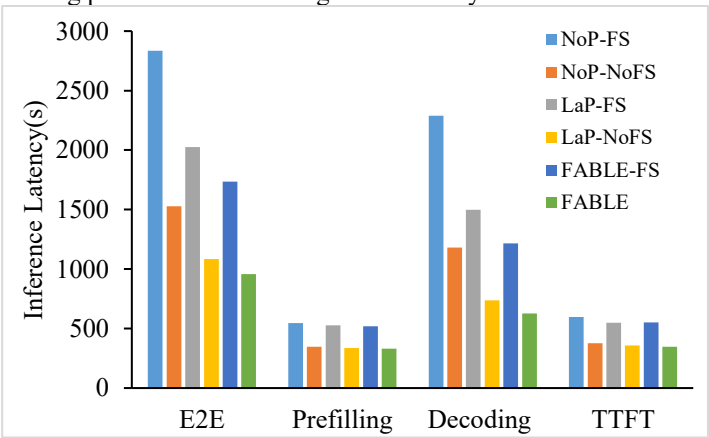


Figure 15. End-to-end inference latency (2048-token input, batch size 64)

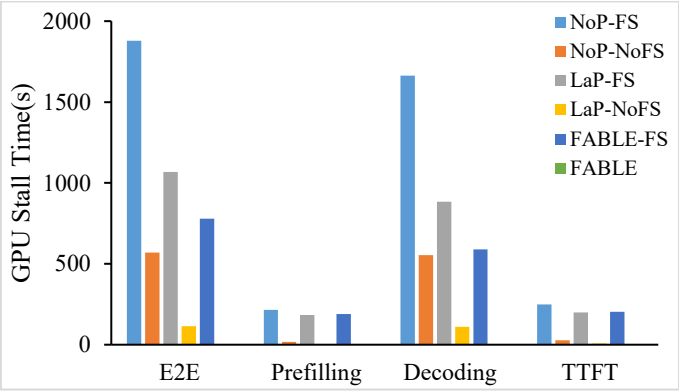


Figure 16. End-to-end inference I/O stall Time (2048-token input, batch size 64)

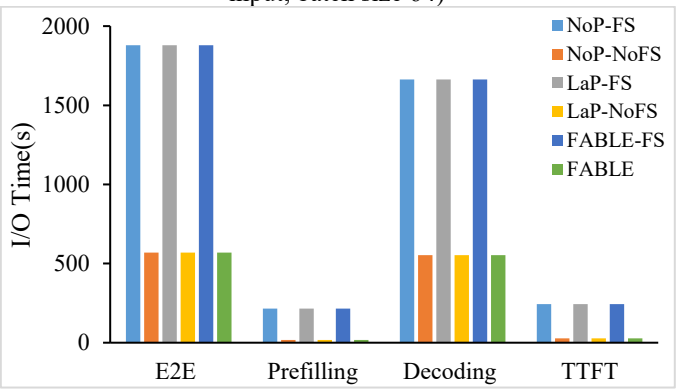


Figure 17. End-to-end inference I/O Time (2048-token input, batch size 64)

End-to-end inference latency. End-to-end latency measures the wall-clock time from the start of prompt prefilling to the return of the final output token, spanning both prefilling and decoding. This measurement excludes one-time setup costs such as model construction/loading and warm-up runs. To connect performance outcomes to our two design choices, this section analyzes end-to-end latency together with its breakdown into prefilling time, decoding time, and time to first token (TTFT) for each system under the same workload (2048-token input, batch size 64, generate token 32) in Figure 15. These metrics reveal the impact of fine-grained prefetching, which improves compute–I/O overlap and reduces GPU stalls, and filesystem bypass, which lowers the cost of SSD to DRAM transfers. Figure 15 shows that the FABLE configuration combines both design choices and reduces end-to-end latency by 66.2% relative to the NoP-FS. This improvement is driven primarily by decoding because decoding latency decreases by 72.6% under the same comparison, indicating that FABLE removes a large fraction of decode-time waiting that dominates the critical path. Prefilling also improves, but prefilling latency decreases by 39.4% only, consistent with prefilling having a larger computation that limits how much faster the phase can become. TTFT improves as well, decreases by 41.6%. The following analysis disentangles the sources of these gains, beginning with the overlap benefit enabled by fine-grained prefetching.

To isolate the effect of scheduling under a fixed storage path, Figure 15 compares FABLE-FS with LaP-FS. Importantly, LaP-FS remains layer-granularity prefetching, but we set its look-ahead prefetch depth to match FABLE-FS under the same filesystem-based SSD path; thus, this comparison factors out any advantage from a deeper prefetch window and isolates the benefit of FABLE's sublayer-granularity pipeline in improving overlap. FABLE-FS reduces end-to-end latency by 11.6%, and the improvement comes mainly from decoding. Under the same filesystem path, decoding latency drops by 17.6%, consistent with fine-grained pipelining reducing decode-side idle time more effectively than layer-granularity prefetching. In contrast, prefilling changes little, with a 1.4% reduction, reflecting that prefilling is less overlap-limited in this setting. TTFT is also nearly unchanged, indicating that sublayer pipelining primarily improves steady-state decoding rather than the first-token boundary. Figure 16 shows GPU stall time, defined as the exposed portion of data movement that remains on the critical path as GPU idle time while waiting for required weight or KV inputs, measured under the same workload and experimental settings as Figure 15. Under FS, moving from NoP-FS (no prefetching) to LaP-FS (layer-level prefetching) reduces end-to-end GPU stall by 43.2%, showing that even coarse, layer-granularity prefetching can hide a substantial fraction of transfer time. Since LaP-FS uses the same prefetch depth as FABLE-FS on the same FS path, the additional reduction from FABLE-FS reflects the benefit of sublayer-granularity prefetching and pipelining rather than a deeper prefetch window. FABLE-FS then reduces end-to-end GPU stall by a further 27.1% relative to LaP-FS, despite using the same storage path and moving the same amount of data. This remaining gap indicates that finer-grained prefetching is more effective than layer-granularity prefetching at shrinking the residual transfer tail and keeping the GPU busy.

Figure 15 shows that FABLE reduces end-to-end latency by 11.1% relative to LaP-NoFS even when both use the same filesystem-bypass path. Under the same comparison, decoding latency decreases by 15.9%, while prefilling latency decreases by 1.1% and TTFT decreases by 1.4%. This result indicates that faster I/O alone does not eliminate stalls. Figure 16 explains the reason. Under NoFS, transfers are cheaper and more predictable, so overlap becomes easier for all schedulers. However, LaP's layer-granularity prefetching still leaves residual GPU idle time when the remaining transfer tail cannot be fully hidden behind per-layer compute. In contrast, FABLE's sublayer-granularity pipeline increases overlap opportunities by prefetching and staging at the fine-grained level and launching each sublayer as soon as its own inputs are ready. As a result, LaP-NoFS retains a non-zero end-to-end stall component, whereas FABLE drives end-to-end stall to near zero within measurement granularity, showing that fine-grained scheduling can more completely hide the remaining transfers even on a fast NoFS path.

To isolate the storage-path contribution, scheduling is held constant and FABLE-FS is compared against FABLE. Switching from conventional I/O to NoFS reduces end-to-end latency by 44.8%, and this improvement can be attributed to the SSD to DRAM path rather than to scheduling. NoFS bypasses the page cache, which removes page-cache management overhead and reduces the cost and variability of SSD reads into the runtime's pinned staging buffers. The reduction appears in both phases, decoding benefits most. With decode latency decreasing by 48.5% because decoding repeatedly stages weights/KV with limited per-step slack, so any storage-stack

overhead is more likely to surface as GPU waiting. Prefilling latency drops by 36.4% and TTFT drops by 36.9%.

Figure 17 reports total I/O time, defined as the cumulative time spent moving weights and KV across the SSD to DRAM and DRAM to GPU transfer paths, regardless of whether this time is overlapped with computation or appears as stall, measured under the same workload and experimental settings as Figure 15. Switching from FS to NoFS reduces total I/O time by 69.6% overall, indicating a lower per-byte movement cost rather than a change in overlap alone. The reduction holds in both phases. Prefilling total I/O time decreases by 92.3%, and decoding total I/O time decreases by 66.7%. TTFT total I/O time decreases by 88.5%. These percentages look large partly because prefilling I/O becomes very small under NoFS, so the relative reduction is amplified by a small denominator; decoding still incurs repeated transfers across many steps and therefore shows a smaller percentage drop. These results match the behavior of direct I/O, which performs I/O directly to and from user-space buffers under alignment constraints, avoiding page-cache buffering and extra copies when the runtime already manages its own staging buffers.

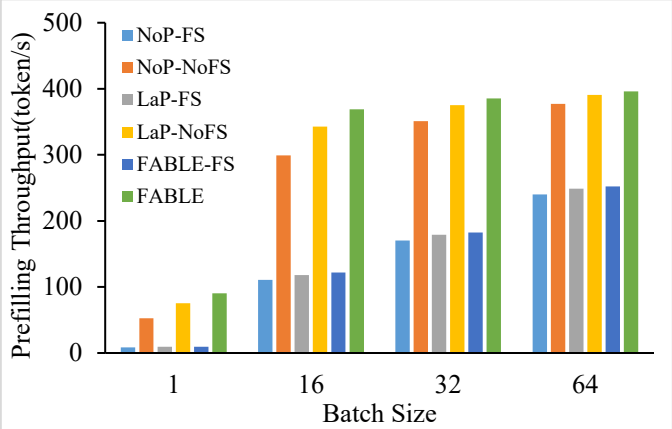


Figure 18. Different batch impact on prefilling throughput

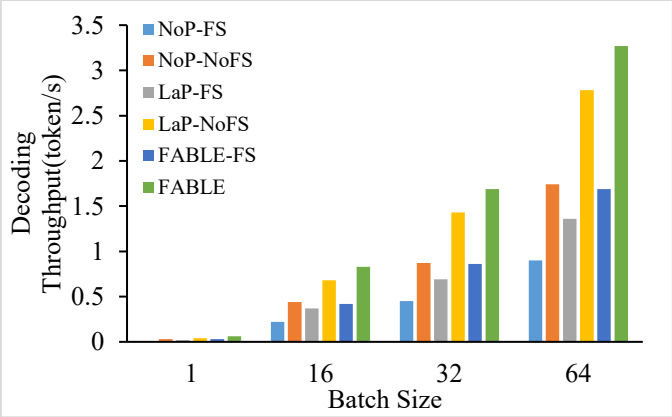


Figure 19. Different batch impact on decoding throughput

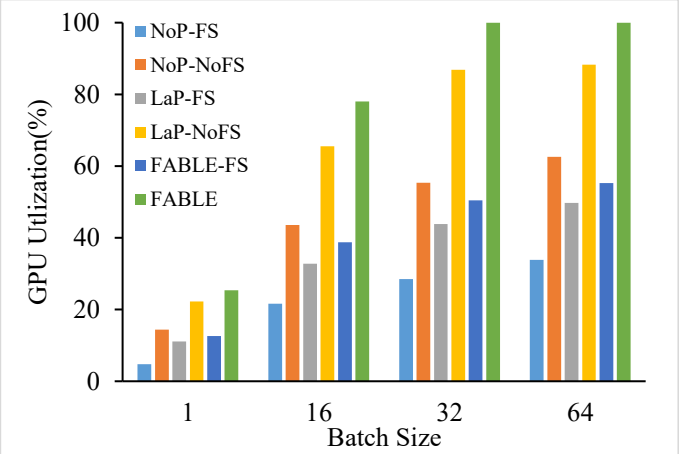


Figure 20. Different batch impact on GPU utilization

Prefilling Under the high-concurrency setting of Figure 15, FABLE reduces prefill latency by 39.4%/4.7%/37.2%/1.4%/36.4% relative to NoP-FS/NoP-NoFS/LaP-FS/LaP-NoFS/FABLE-FS, and across NoFS variants prefill time already converges (within ~5%), indicating limited headroom from scheduling alone in this phase. Prefilling throughput in Figure 18 shows the same convergence at high batch: with NoFS held constant, FABLE improves prefill throughput over NoP-NoFS by 1.72×/1.23×/1.10×/1.05× and over LaP-NoFS by 1.20×/1.08×/1.03×/1.01× at batch 1/16/32/64. The storage path dominates movement cost in prefill. Figure 17 shows that switching from FS to NoFS reduces prefill total I/O time by 92.3%. The remaining differences are explained by overlap quality in Figure 16. Under the FS path, both LaP-FS and FABLE-FS reduce prefilling stall only slightly, by 9.1% and 12.3% relative to NoP-FS, respectively. NoP-FS serves as the reference because it exposes nearly all prefilling data movement as stall under on-demand execution, making it a proxy for the full prefilling I/O waiting cost on this path. Under the NoFS path, overlap becomes much more effective: LaP-NoFS reduces prefilling stall by 72.5% relative to NoP-NoFS, and FABLE further drives the prefilling stall component to near zero.

As Figure 18 illustrate, with the storage path fixed to NoFS, prefilling throughput differs little at high batch. At batch 64, FABLE improves prefilling throughput by only 1.01× over LaP-NoFS and 1.05× over NoP-NoFS. With filesystem overhead removed, the remaining prefilling workload keeps the GPU busy enough that prefilling is dominated by compute, leaving little exposed stall for scheduling to eliminate. This convergence is also reflected in Figure 20. Following the nvidia-smi/NVML [41] definition, we measure GPU utilization as the percentage of the end-to-end inference interval during which one or more kernels are executing on the GPU, which directly reflects idle gaps caused by waiting for weights or KV blocks to become ready. As batch size increases, GPU utilization rises across all designs under the NoFS path, but the degree of saturation differs. At batch 32, GPU utilization is 55.4% for NoP-NoFS, 86.8% for LaP-NoFS, and 99.9% for FABLE. At batch 64, it increases to 62.6% for NoP-NoFS, 88.3% for LaP-NoFS, and remains 99.9% for FABLE. These numbers show that FABLE is already near compute saturation at high batch, so additional scheduling gains have limited headroom,

which is consistent with the shrinking prefill differences among NoFS variants at high batch. Fine-grained prefetching still helps at low batch, where the compute window is short and on-demand execution frequently follows a load-then-run pattern that leaves the GPU waiting for inputs. FABLE's sliding-window lookahead stages near-future objects earlier and reduces the exposed stall component. Consistent with this reduction, at batch 1 FABLE improves prefilling throughput by 1.20× over LaP-NoFS and by 1.72× over NoP-NoFS in Figure 18.

Decoding. Figure 15 shows FABLE's improvement is decode-dominated. At batch 64, FABLE reduces decoding latency by 72.6%/46.9%/58.4%/15.0%/48.5% relative to the NoP-FS/NoP-NoFS/LaP-FS/LaP-NoFS/FABLE-FS, respectively. Decode throughput preserves the same ordering across batches when the path is fixed: with NoFS held constant, FABLE improves decode throughput over NoP-NoFS by 2.00×/1.89×/1.94×/1.88× and over LaP-NoFS by 1.50×/1.22×/1.18×/1.15× at batch 1/16/32/64, respectively in Figure 19

In decoding, switching FS to NoFS substantially reduces total I/O time, indicating that the storage path is a key factor in data-movement cost. Figure 17 shows that switching from FS to NoFS reduces decode total I/O time by 66.7%, while under a fixed path the total I/O time is invariant across schedulers, so decode separation is primarily driven by I/O stall (overlap quality). Figure 17 confirms this: under FS, LaP-FS reduces decode stall by 45.3% relative to NoP-FS, while FABLE-FS reduces decode stall by 64.6% relative to NoP-FS and by an additional 35.3% relative to LaP-FS. Under NoFS, LaP-NoFS reduces decode stall by 70.1% relative to NoP-NoFS, while FABLE drives the remaining stall component to near-zero, indicating near-complete overlap of transfers with GPU kernels. Since Figure 17 shows the same total I/O time under a fixed path across schedulers, these stall gaps reflect purely how much movement remains exposed as bubbles: LaP overlaps at layer granularity so any transfer tail beyond per-layer slack remains exposed, whereas FABLE's finer-grained prefetch increases overlap opportunities and shrinks the residual tail. Consistent with the reduced exposed bubbles, Figure 19 shows large utilization gaps at throughput-oriented batches: at batch 32/64, NoP-NoFS achieves 55.4%/62.6% GPU utilization, LaP-NoFS improves to 86.8%/88.3%, while FABLE is essentially saturated at 99.9%/99.9%

Time to first token (TTFT). TTFT measures the user-visible delay from when a request enters the engine, and prefilling begins to when the first output token is produced. In our setup, TTFT includes the entire prefilling pass over the prompt and the first decoding step. Prompt processing uses chunked prefilling, so the engine does not emit any output tokens until all prompt chunks have been processed; token emission occurs in the first decoding step. Accordingly, TTFT includes this initial decoding step that produces the first token. As shown in Figure 15, FABLE reduces TTFT by 41.6% relative to NoP-FS, and that most of this gain comes from the storage path: switching from FABLE-FS to FABLE reduces TTFT by 36.9% under the same scheduler. This explanation is corroborated by Figure 17, which shows that switching from FS to NoFS reduces the total I/O time along the TTFT critical path by 88.5%, indicating that TTFT is largely shaped by storage-stack overhead before the first token can be emitted. Under a fixed path, the remaining TTFT differences are therefore governed by overlap. Figure 16 shows that under FS only a modest fraction of TTFT waiting can be hidden, whereas under NoFS the residual stall becomes small and FABLE drives it to near-zero, which is why TTFT margins across NoFS schedulers naturally shrink, for example to 2.1% relative to LaP-NoFS.

4.3. Object Budget Sensitivity to Prefetch Depth

Figure 21 and Figure 22 show how sliding-window lookahead affects end-to-end latency by varying two capacity budgets: GPU MAX object, which bounds the number of objects staged in GPU memory, and CPU MAX object, which bounds the number of objects cached in host DRAM. In this paper, we treat the per-layer KV state as part of the MHA object and manage it under the same object window. Therefore, object movement includes both MHA/FFN weights and (for MHA) the bundled KV state blocks. GPU MAX object sets the lookahead depth for on-GPU residency. A larger window increases the chance that DRAM to GPU memory transfers overlap with compute, reducing exposed stalls. CPU MAX object determines how much near-future working set can be retained in DRAM, which reduces SSD to DRAM promotions and makes performance less sensitive to SSD-side latency variation. Thus, increasing GPU MAX object increases the number of upcoming sublayers whose MHA/FFN objects (and for MHA, their KV state blocks) can be kept resident and ready to run. Under Llama 3.1 70B on FABLE 16GB GPU memory and 128GB DRAM budget, we observe hard capacity ceilings of 12 objects in GPU memory and 94 objects in DRAM and use them as default window sizes. This choice maximizes lookahead while remaining feasible and avoids forced evictions that would reintroduce reload stalls. Unless otherwise noted, we vary one budget at a time while holding the other fixed: for the GPU-window sweep in Figure 21, we fix CPU MAX object to the 96-object setting (our default DRAM window), and for the DRAM-window sweep in Figure 22 we fix GPU MAX object to 12 (our default GPU window).

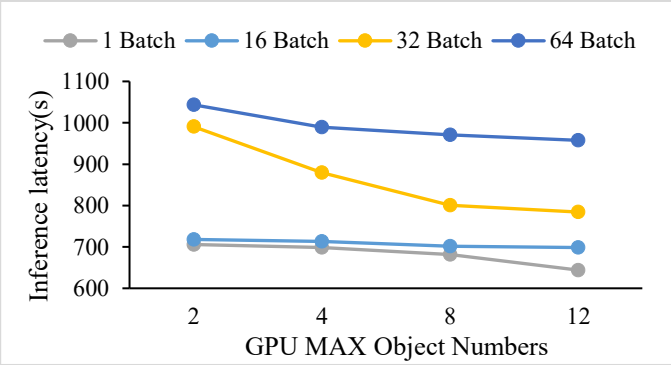


Figure 21. Different number of GPU MAX objects impact on e2e inference latency

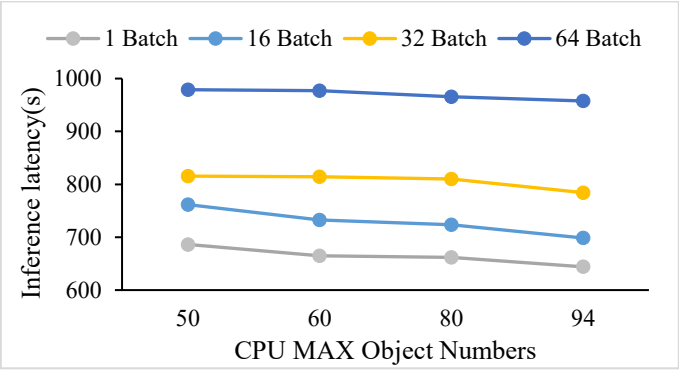


Figure 22. Different number of CPU MAX objects impact on e2e inference latency

The GPU memory window primarily controls whether object movement is hidden by computation or exposed as end-to-end latency in Figure 21. Concretely, a larger GPU window keeps more upcoming MHA/FFN objects resident on the GPU, increasing the GPU hit rate and reducing repeated DRAM to GPU memory reloads. When batch size provides sufficient compute slack, the remaining transfers can be largely overlapped; when slack is limited (e.g., small batches), the non-overlapped tail is exposed as bubbles (idle gaps) between compute segments. This is also where FABLE differs most from layer-granularity baselines: NoP on-demand execution provides essentially no lookahead and thus frequently blocks computation on data availability, while FlexGen prefetches weights at layer granularity and overlaps load next-layer weights with computation current, but its overlap opportunities remain bounded by layer dependencies. Consistent with this mechanism, shrinking the GPU memory window measurably increases latency in a batch-dependent manner. At batch size 32, Increasing GPU MAX objects from 2 to 12 reduces e2e latency by 20.82%, because a larger window avoids repeated reloads and provides enough slack to overlap most weight transfers. Under the same change from 2 to 12 objects, the reduction is smaller at batch sizes 1 and 16, at 8.76% and 2.73%, respectively, indicating that execution remains more transfer-limited and gains less from additional lookahead. At batch size 64, the benefit saturates once the window reaches roughly 4–8 objects. Increasing the window further to 12 yields only an additional 1.35% reduction, suggesting that most weight-transfer latency that can be hidden by lookahead is already covered at this batch size. The remaining gap is then dominated by bandwidth-sensitive decode-time state movement that grows with batch. Specifically, the per-step traffic of KV state blocks within MHA objects over the host–device interconnect. This state footprint grows linearly with batch size and sequence length, and the CPU–GPU link can become the limiting factor under offloading.

The DRAM window mainly affects how often the runtime must promote objects from SSD into DRAM and how robust it is to SSD-side variability in Figure 22. CPU MAX object bounds how many near-future MHA/FFN objects FABLE can retain in host DRAM; increasing this window raises the DRAM hit rate and directly reduces SSD to DRAM promotions triggered by capacity misses. Fewer promotions shorten the exposed I/O component on the critical path and reduce exposure to SSD-latency variation, so end-to-end latency decreases until the near-future working set largely fits in DRAM, after which returns diminish. This effect is most visible at batch 16, where increasing CPU MAX object from 50 to 94 reduces end-to-end latency by up to 11.76%, indicating that SSD promotions are frequent enough to matter and that DRAM caching effectively removes them. At higher concurrency (e.g., batch 64), the gain shrinks to 0.85–1.98%, consistent with the bottleneck shifting away from SSD promotions toward decode-time KV state traffic within MHA objects and the CPU–GPU interconnect. As noted above, KV state grows linearly with batch size and sequence length, and under offloading the limiting factor commonly shifts toward host–device bandwidth rather than SSD read depth. Overall, the DRAM window primarily improves performance by reducing how often the engine must fetch objects from SSD, while its impact saturates once SSD promotion is no longer on the critical path under extreme concurrency.

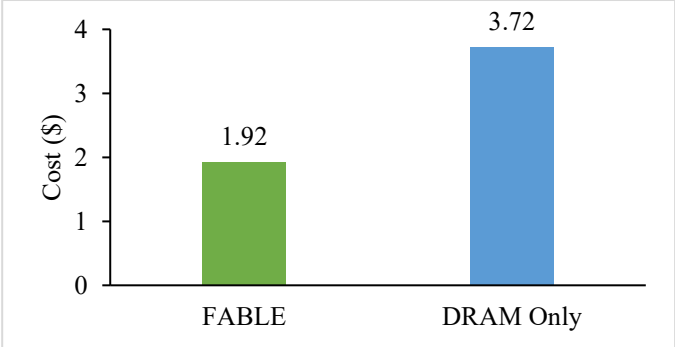


Figure 23. Estimated AWS on-demand cost comparison between FABLE and a DRAM-only baseline.

Inference cost. We evaluate cost by comparing FABLE's SSD-backed hierarchy against a DRAM-only alternative that eliminates SSD and instead provisions enough host DRAM to hold the offloaded model state required to complete the workload. This DRAM-only configuration is not feasible on our local testbed and fails with host-memory out-of-memory errors; however, it represents the natural capacity-scaling baseline in practice, i.e., buy more DRAM to avoid SSD. Figure 23shows that FABLE costs $1.92 per workload, whereas the DRAM-only baseline costs $3.72, yielding a 48.4% reduction. The dominant driver is capacity cost. Under AWS pricing, SSD capacity is substantially cheaper than host DRAM per GiB-hour, so shifting the bulk of capacity from DRAM to SSD materially reduces the storage bill. Crucially, FABLE avoids turning this cheaper hierarchy into a proportional increase in GPU time by combining a filesystem-bypass I/O path that reads/writes directly to aligned user-space buffers with computer-I/O overlap that pipelines SSD to DRAM staging and DRAM to GPU transfers with execution.

5. CONCLUSIONS AND FUTURE WORK

5.1. Conclusion

This paper presented FABLE, a hierarchical-storage inference runtime that enables cost-efficient, high-concurrency serving of 70B-scale LLMs on a single consumer GPU. FABLE makes offloaded inference schedulable by separating computation from data movement: it executes each Transformer layer as two sublayer compute units (MHA and FFN) and stages their required payloads as objects. In particular, an MHA object

bundles the attention weights together with the per-request KV blocks, while an FFN object contains only FFN weights. Using a sliding-window controller and CUDA streams/events, FABLE overlaps multi-tier movement of object with sublayer execution, reducing GPU "wait-for-data" bubbles. In addition, FABLE reduces SSD to DRAM overhead and latency jitter with NoFS, a filesystem-bypass direct-I/O path that bypasses the OS page cache.

On a single RTX 5080 serving FP16/BF16 Meta Llama 3.1 70B with 2,048-token prompts and batch sizes up to 64, FABLE reduces end-to-end latency by up to 66.2% versus a on-demand offloading baseline using buffered filesystem I/O, and by 15.3% versus a strong layer-level overlap baseline under the same NoFS path (FlexGen-style). With the NoFS data path held constant, FABLE improves decoding throughput at batch size 64 by 1.88× over on-demand offloading and by 1.27× over layer-level overlap, while achieving near-saturated GPU utilization (up to ~99%) in throughput-oriented settings. Finally, under a GPU-hour–based cloud-style cost model, FABLE reduces the total inference cost from $3.72 to $1.92 by relying on SSD as the primary capacity tier without incurring a proportional performance penalty.

5.2. Future Work

We plan to integrate FABLE with mainstream LLM serving stacks such as vLLM and SGLang, so that FABLE complements their mature request scheduling and serving interfaces instead of duplicating them. vLLM combines continuous batching with PagedAttention, which manages GPU-resident KV cache in paged (block-based) form for high throughput. SGLang co-designs a frontend language with a runtime that exploits RadixAttention for efficient prefix reuse across multi-call or repeated-prefix workloads. Our goal is to expose FABLE as a pluggable hierarchical-storage backend that provides out-of-core weight and KV capacity across SSD and DRAM together with the low-overhead NoFS I/O path, while leaving batching, routing, and application-level scheduling to the host framework.

The main technical work is to define clean abstraction boundaries. First, we will align FABLE's sliding-window lookahead with continuous batching so that prefetch and eviction reflect the actual token-level execution order chosen by the serving scheduler. Second, we will reconcile FABLE's KV block layout and tiering with existing KV representations—such as paged KV allocation in vLLM and prefix-aware KV reuse in SGLang—so GPU-side fast paths remain effective while FABLE supplies a low-cost capacity tier for overflow, long sessions, and large working sets. Finally, we will maintain FABLE as a long-lived framework by enforcing clear module boundaries between the storage path (NoFS), caching/prefetch policy, and execution pipeline, and by providing correctness tests, performance regression tests, and reproducible benchmarks that track upstream model and kernel evolution.

REFERENCES

[1] A. Vaswani *et al.*, "Attention Is All You Need," Aug. 02, 2023, *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.

[2] " ChatGPT." Accessed: Dec. 09, 2025. [Online]. Available: https://openai.com/zh-Hans-CN/index/chatgpt/

[3] Y. Sheng *et al.*, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," June 12, 2023, *arXiv*: arXiv:2303.06865. doi: 10.48550/arXiv.2303.06865.

[4] B. Gao *et al.*, "Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention".

[5] R. Y. Aminabadi *et al.*, "DeepSpeed- Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, Dallas, TX, USA: IEEE, Nov. 2022, pp. 1–15. doi: 10.1109/SC41404.2022.00051.

[6] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, Koblenz Germany: ACM, Oct. 2023, pp. 611–626. doi: 10.1145/3600006.3613165.

[7] S. Gao, Y. Chen, and J. Shu, "Fast State Restoration in LLM Serving with HCache," in *Proceedings of the Twentieth European Conference on Computer Systems*, Rotterdam Netherlands: ACM, Mar. 2025, pp. 128–143. doi: 10.1145/3689031.3696072.

[8] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FLASHATTENTION: Fast and Memory-Efficient Exact Attention with IO-Awareness".

[9] B. Debnath, S. Sengupta, and J. Li, "FlashStore: high throughput persistent key-value store," *Proc. VLDB Endow.*, vol. 3, no. 1–2, pp. 1414–1425, Sept. 2010, doi: 10.14778/1920841.1921015.

[10] Z. Qureshi *et al.*, "GPU-Initiated On-Demand High-Throughput Storage Access in the BaM System Architecture," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, Jan. 2023, pp. 325–339. doi: 10.1145/3575693.3575748.

[11] Z. Zhang *et al.*, "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models".

[12] J. Peng *et al.*, "Harnessing Your DRAM and SSD for Sustainable and Accessible LLM Inference with Mixed-Precision and Multi-level Caching," Oct. 23, 2024, *arXiv*: arXiv:2410.14740. doi: 10.48550/arXiv.2410.14740.

[13] W. Lee, J. Lee, J. Seo, and J. Sim, "InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache Management".

[14] X. Pan *et al.*, "InstAttention: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Mar. 2025, pp. 1510–1525. doi: 10.1109/HPCA61900.2025.00113.

[15] X. Pan *et al.*, "InstInfer: In-Storage Attention Offloading for Cost-Effective Long-Context LLM Inference," Sept. 08, 2024, *arXiv*: arXiv:2409.04992. doi: 10.48550/arXiv.2409.04992.

[16] C. Hooper *et al.*, "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization".

[17] M. Bjørling, "LightNVM: The Linux Open-Channel SSD Subsystem".

[18] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” July 19, 2023, *arXiv*: arXiv:2307.09288. doi: 10.48550/arXiv.2307.09288.
[19] H. Touvron *et al.*, “LLaMA: Open and Efficient Foundation Language Models,” Feb. 01, 2023, *arXiv*. doi: 10.48550/arXiv.2302.13971.
[20] K. Alizadeh *et al.*, “LLM in a flash: Efficient Large Language Model Inference with Limited Memory,” July 30, 2024, *arXiv*: arXiv:2312.11514. doi: 10.48550/arXiv.2312.11514.
[21] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale”.
[22] A. Gupta, G. Dar, S. Goodman, D. Ciprut, and J. Berant, “Memory-efficient Transformers via Top-k Attention,” June 13, 2021, *arXiv*: arXiv:2106.06899. doi: 10.48550/arXiv.2106.06899.
[23] R. Qin *et al.*, “Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving,” *ACM Trans. Storage*, p. 3773772, Nov. 2025, doi: 10.1145/3773772.
[24] G.-I. Yu, J. S. Jeong, G.-W. Kim, and B.-G. Chun, “Orca: A Distributed Serving System for Transformer-Based Generative Models”.
[25] Y. Bisk, R. Zellers, R. Le Bras, J. Gao, and Y. Choi, “PIQA: Reasoning about Physical Commonsense in Natural Language,” *AAAI*, vol. 34, no. 05, pp. 7432–7439, Apr. 2020, doi: 10.1609/aaai.v34i05.6239.
[26] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” Dec. 03, 2019, *arXiv*: arXiv:1912.01703. doi: 10.48550/arXiv.1912.01703.
[27] L. Zheng *et al.*, “SGLang: Efficient Execution of Structured Language Model Programs,” June 06, 2024, *arXiv*: arXiv:2312.07104. doi: 10.48550/arXiv.2312.07104.
[28] “Storage Performance Development Kit.” Accessed: Dec. 10, 2025. [Online]. Available: https://spdk.io/
[29] Y. Wang, K. Chen, H. Tan, and K. Guo, “Tabi: An Efficient Multi-Level Inference System for Large Language Models,” in *Proceedings of the Eighteenth European Conference on Computer Systems*, Rome Italy: ACM, May 2023, pp. 233–248. doi: 10.1145/3552326.3587438.
[30] K. Han, A. Xiao, E. Wu, J. Guo, C. XU, and Y. Wang, “Transformer in Transformer,” in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2021, pp. 15908–15919. Accessed: Dec. 10, 2025. [Online].
[31] “NVIDIA A100 GPUs Power the Modern Data Center,” NVIDIA. Accessed: Dec. 10, 2025. [Online]. Available: https://www.nvidia.com/en-us/data-center/a100/
[32] “NVIDIA GeForce RTX 5080 Graphics Cards,” NVIDIA. Accessed: Dec. 10, 2025. [Online].
[33] “Assistants migration guide | OpenAI API.” Accessed: Dec. 10, 2025. [Online].
[34] S. Zhang et al., “OPT: Open Pre-trained Transformer Language Models,” June 21, 2022, arXiv: arXiv:2205.01068. doi: 10.48550/arXiv.2205.01068.
[34] S. Liang, Y. Wang, Y. Lu, Z. Yang, H. Li, and X. Li, “Cognitive SSD: A Deep Learning Engine for In-Storage Data Retrieval,” presented at the 2019 USENIX Annual Technical Conference (USENIX ATC 19), 2019, pp. 395–410. Accessed: Dec. 10, 2025. [Online].
[35] “meta-llama/Llama-3.1-70B · Hugging Face.” Accessed: Dec. 10, 2025. [Online].
[36] S. Kim, G. Lee, J. Woo, and J. Jeong, “Zero-Copying I/O Stack for Low-Latency SSDs,” IEEE Computer Architecture Letters, vol. 20, no. 1, pp. 50–53, Jan. 2021, doi: 10.1109/LCA.2021.3064876.
[37] A. Grattafiori et al., “The Llama 3 Herd of Models,” Nov. 23, 2024, arXiv: arXiv:2407.21783. doi: 10.48550/arXiv.2407.21783.
[38] G. Team et al., “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context,” Dec. 16, 2024, arXiv: arXiv:2403.05530. doi: 10.48550/arXiv.2403.05530.Comment: This is an extension to the published conference paper at ASPLOS'23: https://dl.acm.org/doi/abs/10.1145/3575693.3575748
[39] “ShareGPT: Share your wildest ChatGPT conversations with one click.” Accessed: Dec. 17, 2025. [Online]. Available: https://sharegpt.com
[40] “AWS Pricing,” Amazon Web Services, Inc. Accessed: Dec. 24, 2025. [Online].
[41] “NVIDIA Management Library (NVML),” NVIDIA Developer. Accessed: Dec. 26, 2025. [Online]. Available: https://developer.nvidia.com/management-library-nvml
[42] J. Liu et al., “A Comprehensive Survey on Long Context Language Modeling,” Nov. 24, 2025, arXiv: arXiv:2503.17407. doi: 10.48550/arXiv.2503.17407.
[43] A. Agrawal et al., “Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve,” presented at the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024, pp. 117–134. Accessed: Jan. 03, 2026. [Online].
[44] Y. Zhong et al., “DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving,” presented at the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024, pp. 193–210. Accessed: Jan. 03, 2026. [Online].
[45] L. Gao, C. Jiang, H. E. Zarch, D. Wong, and M. Annavaram, “DuetServe: Harmonizing Prefill and Decode for LLM Serving via Adaptive GPU Multiplexing,” Nov. 06, 2025, arXiv: arXiv:2511.04791. doi: 10.48550/arXiv.2511.04791.
[46] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” May 13, 2020, arXiv: arXiv:1910.02054. doi: 10.48550/arXiv.1910.02054.
[47] Y. Fu et al., “ServerlessLLM: Low-Latency Serverless Inference for Large Language Models,” presented at the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), 2024, pp. 135–153. Accessed: Jan. 04, 2026. [Online].
[48] “Introducing NVIDIA BlueField-4-Powered Inference Context Memory Storage Platform for the Next Frontier of AI,” NVIDIA Technical Blog. Accessed: Jan. 10, 2026. [Online].
[49] “Hugging Face – The AI community building the future.” Accessed: Feb. 01, 2026. [Online]. Available: https://huggingface.co/